BACHELOR'S DEGREE IN SOFTWARE ENGINEERING

COOPERATIVE SYSTEM OF COMMUNICATION BETWEEN ROBOTS

TRABAJO FIN DE GRADO --- CONVOCATORIA ORDINARIA 2026

Author: **Raquel Suárez Sánchez**
UO295000@uniovi.es
54472472P

Tutors: **Germán León Fernández**
Olaya Pérez Mon

DECLARATION OF AUTHORSHIP AND ORIGINALITY OF THE FINAL DEGREE PROJECT

In accordance with the provisions of Article 8.3 of the Agreement of July 17, 2020, of the Governing Council of the University of Oviedo, which approves the Regulations on the preparation and defense of final master's projects at the University of Oviedo (BOPA No. 153 of August 7, 2020).

Mrs. **Raquel Suárez Sánchez**, with ID **54472472P**

I DECLARE:

The Final Degree Project entitled **"Cooperative System of Communication Between Robots"**, which I present for its exposition and defense, is original and I have duly cited all the sources of information used, both in the body of the text and in the bibliography.

Firmado por SUAREZ SANCHEZ RAQUEL - ***7247** el día 30/05/2026 con un certificado emitido por AC FNMT Usuarios

In Oviedo, 29th of May 2026

Acknowledgments

Thank you to my tutors, Olaya and Germán, for their patience and availability in this process.

I would like to say a special thank you to my parents, my sister and the rest of my family, for supporting me so I could focus on my studies.

To the friends this degree gave me, I am so lucky to have a group of smart and hard-working women who I look up to. I've had so much fun these past 4 years. This achievement is yours too.

To the rest of my friends, thank you for making me laugh and giving me a place to disconnect, this wouldn't have been possible without you.

Finally, to myself, for all the hours of work I have dedicated to this degree. Closing this important stage in my life, excited for what comes next.

Table of contents

Table Index

Figure Index

Chapter 1. General Description of the project

1.1. Resumen

Las futuras misiones lunares dependerán cada vez más de enjambres robóticos autónomos para explorar de forma eficiente entornos extensos y hostiles. Por ello, mantener una comunicación estable en terrenos irregulares es un desafío crucial, ya que los activos pueden perder fácilmente la conexión con la estación base, con el consecuente riesgo de pérdida de datos. Este trabajo de fin de grado presenta el diseño y desarrollo de un entorno de simulación de software para evaluar la coordinación y el enrutamiento dinámico de la red de un enjambre robótico lunar.

El sistema utiliza datos topográficos reales para crear una cuadrícula de navegación y simula el intercambio de telemetría mediante sockets *UDP (User Datagram Protocol)*. Un algoritmo de enrutamiento dinámico permite que los rovers actúen como repetidores de señal en función de la proximidad y el nivel de batería. Además, la simulación evalúa dos estrategias de movimiento (exploración libre y exploración fija) para analizar su impacto en la cobertura total del terreno y la estabilidad. En definitiva, esta herramienta proporciona un entorno virtual robusto y escalable para analizar el comportamiento de enjambres, desarrollado en el marco del proyecto *"Communications System for Lunar Surface and Subsurface Exploration" (COSLE)* de la Agencia Espacial Europea (ESA).

1.2. Palabras clave

Misión de exploración, Activos, Flota de robot, Sistemas distribuidos, Redes Ad-hoc móviles (MANET), Enrutamiento dinámico

1.3. Abstract

Future lunar missions will increasingly rely on autonomous robotic swarms to efficiently explore extensive and harsh environments. However, maintaining a stable communication across irregular terrain is a critical challenge, as rovers can easily lose connection with the base station, risking the loss of data. This Bachelor Thesis presents the design and development of a software simulation environment to evaluate the coordination and dynamic network routing of a lunar robotic swarm.

The system utilizes real topographic elevation data to create a navigational grid and simulates telemetry exchange using *UDP (User Datagram Protocol)* sockets. A dynamic routing algorithm allows rovers to act as signal relays based on proximity and battery levels. Furthermore, the simulation evaluates two movement strategies (Free and Fixed exploration) to analyze their impact on total terrain coverage and stability. Ultimately, this tool provides a robust and scalable virtual environment for analyzing swarm behavior, developed within the framework of the European Space Agency's (ESA) *"Communications System for Lunar Surface and Subsurface Exploration" (COSLE)* project.

1.4. Keywords

Scouting Mission, Assets, Swarm Robotics, Distributed systems, Mobile Ad-hoc Networks (MANET), Dynamic routing

Chapter 2. Introduction

2.1. Objective

In recent times, especially this last year, space exploration has experienced a significant revival, with a particular focus on returning to the Moon. Future lunar missions are increasingly relying on autonomous robotic swarms, this offers redundancy, parallel task execution, and the ability to cover vast areas of the lunar surface in less time. However, operating multiple robots simultaneously in a harsh environment like the Moon, introduces many technical challenges.

One of the most critical issues in swarm robotics is maintaining a stable communication network, if an explorer robot travels too far from the base station or loses signal, the scientific data it collects may be lost. Therefore, it is necessary to implement protocols that allow robots to act as relays for one another, ensuring that every node maintains a continuous link to a Supervisor (SV) node while exploring.

The primary objective of this End of Degree Project is to design and develop a software-based simulation environment that models the coordination and communication of a robotic swarm. This project aims to simulate, taking into account realistic communications constraints, how robots exchange telemetry and dynamically rebuild their communication routing trees as they move. Furthermore, the project seeks to evaluate different exploration strategies (Free and Fixed exploration) to analyze how rules impact the total area discovered and the stability of the network.

To achieve this objective, we will follow a set of activities or steps, following a cascade methodology: Start and planification, Research and scope of the project, Analysis, Design, Implementation, Testing and validation, and finally Documentation & closing. The timeline will be later specified on section 3.1.4. Initial plan and WBS (Work Breakdown Structure), including the time dedicated to the weekly meeting.

This bachelor's thesis has been developed within the framework of the project "Communication System for Lunar Surface and Subsurface Exploration (COSLE)", funded by the European Space Agency (ESA), in which the Universidad de Oviedo participates.

2.2. System scope

The scope of this project encompasses the design, development, and validation of a software-based simulation environment to evaluate the coordination and communication of a swarm of lunar robots. The system focuses on routing topologies and exploration coverage within a realistic topographic context by a given map.

In-Scope:

- **System Configuration:** A graphical user interface (GUI) to define initial parameters, such as the number of rovers, starting coordinates, exploration area, and movement strategies, supporting both manual input and automated JSON file uploads.
- **Terrain Processing:** The ability to load, interpret, and represent real elevation data from .dt2 files, transforming GPS coordinates into a 2D grid.
- **Network Simulation:** A functional, decentralized communication model utilizing local UDP sockets to simulate telemetry exchange. It includes a dynamic routing algorithm (Breadth-First Search) that calculates the optimal connection paths to ensure all explorer nodes maintain communication with the Supervisor (SV) node.
- **Exploration Logic:** The implementation of two distinct movement strategies (*Free Exploration* and *Fixed Exploration*) to evaluate how the swarm covers the terrain under different navigational rules.
- **Real-Time Visualization:** A 2D graphical representation using Matplotlib that continuously updates the assets' positions, the percentage of the map explored, and the active network links between nodes.

Out-of-Scope: To maintain the feasibility of the project and focus on the coordination logic, the following elements are considered out of scope:

- **Advanced Physics Engine:** The simulation is purely in 2D. It does not account for physical interactions such as gravity, wheel friction or terrain collisions
- **3D Rendering:** The environment is visualized as a 2D top-down projection; no 3D modeling or rendering of the rovers or the lunar craters is performed.
- **Hardware Integration:** The project is strictly a software-based simulation. Integration with actual robotic hardware, sensors, or specialized frameworks is not included.

- **Complex Energy Models:** While battery telemetry is transmitted, the simulation does not calculate advanced energy consumption rates based on terrain inclination or temperature.

2.3. User requirements

UR-1. Introducing simulation parameters

UR-1.1: The system must allow the user to manually configure the simulation through a graphical user interface.

UR-1.2: The system must allow the user to define the swarm size, accepting a number of explorer robots (between 3 and 7).

UR-1.3: The system must allow the user to set the initial position of the Surface Vehicle (SV) using relative coordinates (X and Y) with respect to the center of the map.

UR-1.4: The system must allow the user to define the initial size of the exploration area (in meters).

UR-1.5: The system must offer the user the choice between two exploration modes: "Free Exploration Mode" and "Fixed Exploration Mode".

UR-1.6: The system must allow configuration automation by uploading an external file instead of manually entering the data.

UR-1.7: The system must display the parameters introduced by the user during the simulation.

UR-2. Communication between robots

UR-2.1: The system must allow explorer robots to continuously transmit their telemetry data (current position and battery level) to the Surface Vehicle (SV).

UR-2.2: The system must allow the Surface Vehicle (SV) to broadcast its status and commands to all connected explorer robots.

UR-2.3: The system must ensure communication continuity by allowing robots to act as network relays for other robots that are out of direct range of the SV.

UR-2.4: The system must automatically adapt the communication network topology in real-time as robots move and distances between them change.

UR-2.5: The system must immediately detect when one or more robots lose connection (i.e., no available path to the SV exists) and stop the simulation, notifying the user of the disconnected robots.

UR-3. Terrain exploration functioning

UR-3.1: The system must dynamically generate and update an exploration map in real-time based on the areas traversed by the active robots.

UR-3.2: In "Free Exploration Mode," the system must allow the exploration area to expand dynamically as robots reach the edges of the currently mapped zone.

UR-3.3: In "Fixed Exploration Mode," the system must confine the robots within the initial boundaries, forcing them to bounce off the limits to ensure maximum coverage of the area.

UR-3.4: The system must calculate and display the total exploration progress as a percentage in real-time.

UR-3.5: In “Fixed Exploration Mode”, the system must automatically conclude the simulation when the target area is considered fully explored.

UR-3.6: In “Free Exploration Mode”, the system will conclude only when the simulation is canceled by the user or one of the rovers is unable to find a path to SV.

UR-3.7: During the exploration of the terrain on every mode, the system must have a “Stop/Resume” button, to stop the simulation on a specific time and later resume it.

2.4. State of Art

Before initiating the development, several existing simulation frameworks and tools were evaluated to determine the best approach for this project. The following alternatives were considered:

ROS (Robot Operating System) with Gazebo

ROS (Robot Operating System) is an open-source software development kit for robotics applications [1]. It is used widely for teaching robotics and the basis for most robotics research at all levels.

ROS' learning curve is steep and requires significant computational resources. For the scope of this project, which focuses on 2D routing, telemetry exchange, and exploration logic, setting up a full physics environment in Gazebo would have been an over-engineering.

Webots by Cyberbotics

Webots is an open source and multi-platform desktop application used to simulate robots. It provides a complete development environment to model, program and simulate robots. It is widely used in industry, education and research [2].

Webots is built heavily around hardware simulation (sensors, actuators, friction). Additionally, integrating topographical files directly into a terrain and simulating UDP socket communications is highly complex.

NetLogo

NetLogo is a programmable modeling environment for simulating natural and social phenomena. Modelers can give instructions to hundreds or thousands of independent "agents" (in our case the rovers) all operating concurrently [3].

NetLogo is excellent for simple, grid-based logic and visualizing agent movement. However, it lacks modern networking capabilities to simulate packet transmission, as well as not being possible to open raw mapping files.

ESA Research

Additionally, and since this project is under the framework of another funded by the European Space Agency (ESA), it is interesting to mention that there is ongoing research in the field of "AI for Large Fleet Network Management" [4].

This could be an interesting addition to this product (and will be explained further in 8.2. Expansions) if we wanted for the rovers' movement to not be random, have them go to the most interesting points for research on the map. This could save time, and help us get more meaningful data of the terrain.

Selected Approach

After evaluating the alternatives, the decision was made to build a custom, lightweight 2D simulation engine from scratch using Python. We will utilize libraries like *matplotlib*, *rasterio* and *socket* in order to satisfy the user's requests of rendering, map processing and communication.

Chapter 3. Planning and management

3.1. Planning of the project

3.1.1. Stakeholder identification

Next, a list of all people with some kind of interest in the project will be detailed:

1. **Author and developer of the Project: Raquel Suárez Sánchez.** The primary creator of the project, responsible for the end-to-end lifecycle of the system. Main responsibilities include researching existing literature, designing the system, developing, the testing phases, and writing the final document. The author's main goal is to successfully deliver a robust simulation.
2. **Academic advisors: Germán León Fernandez & Olaya Perez Mon.** Responsible for providing academic supervision, feedback and ensuring the project meets the standards required by the university. They act as mentors throughout the development and documentation phases.
3. **Potential End-Users**. Individuals or organizations that could utilize the simulation software for research, development, or educational purposes. In the context of this project, users may include researchers and academic institutions interested in analyzing swarm robotics in lunar terrains. Additional users could also include space agencies or private companies with research on this field, like ESA, DLR or GMW.

3.1.2. Organization Breakdown Structure (OBS)

Here, you will find the organization breakdown structure for the project at hand, seen on **Figure 1. OBS**:

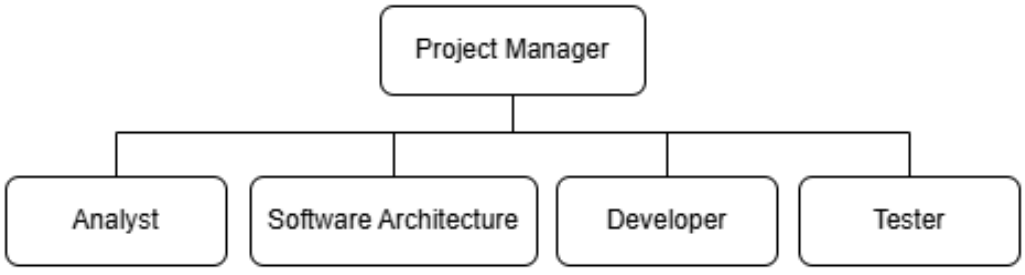

Figure 1. OBS

This project is an individual work; therefore, all roles will be assumed by the author. These defined roles will provide a more natural and realistic look of the project, and will facilitate the planning of the schedule and budget.

For this project, each of the roles will have the following **schedule**:

- Monday from 10.00 to 14.00
- Wednesday from 10.00 to 14.00
- Friday from 10.00 to 15.00

That is a total of 13 hours a week for each member of the team.

3.1.3. Product Breakdown Structure (PBS)

The following is the project's product breakdown structure (**Figure 2. PBS**), representing the main products and deliverables that must be completed until the end:

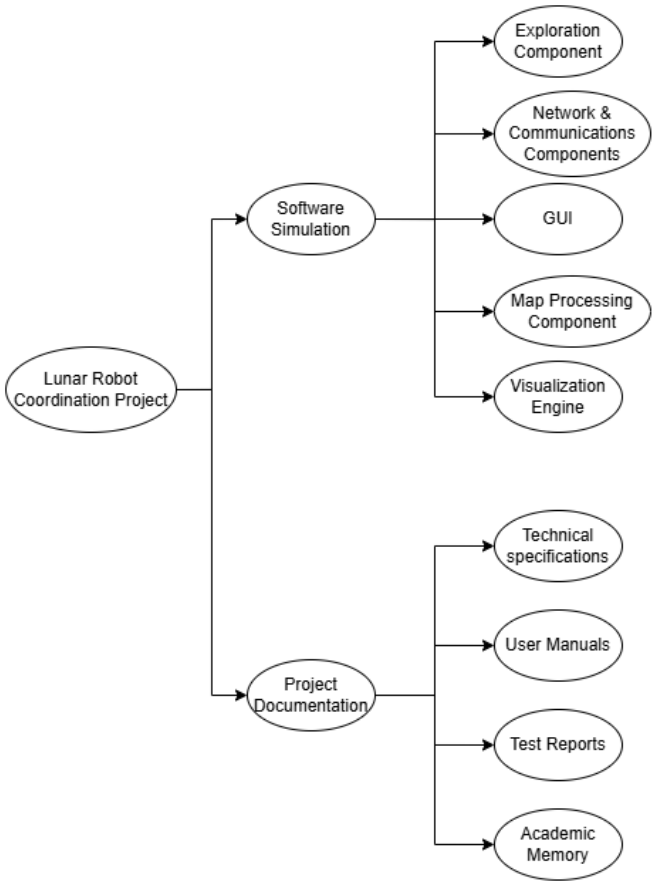

Figure 2. PBS

3.1.4. Initial plan and WBS (Work Breakdown Structure)

In order to estimate the durations in the initial planning, first we divided the project into the following activities: start and planification, research and scope of the project, analysis, design, implementation, testing and validation and finally documentation. Finally, we have an activity solely dedicated to the periodic meetings, which were weekly and lasted for about 30 minutes each.

The total amount of time for this project will be 300 hours, of which each activity will get the following percentages, seen on **Table 1. Work dedicated per activity**:

Activity	Percentage	Time Spent (hrs)
Start and planification	4%	12
Research and scope of the project	7%	21
Analysis	16%	48
Design	17%	51
Implementation	32%	96
Testing and validation	13%	39
Documentation & closing	8%	24
Periodic meeting	3%	9
	TOTAL:	**300 hrs**

Table 1. Work dedicated per activity

These activities and their durations were translated into start and ending dates (seen on **Table 2. WBS**), taking into account working days and the sequential dependencies between tasks:

WBS	Name of task	Work	Start	Finish
1	**Project**	**300 hrs**	**14/01/26**	**20/05/26**
1.1	Periodic meeting	9 hrs	14/01/26	20/05/26
1.2	Start and planification	12 hrs	14/01/26	19/01/26
1.3	Research & Project Scope	21 hrs	19/01/26	30/01/26
1.4	Analysis	48 hrs	30/01/26	16/02/26
1.5	Design	51 hrs	16/02/26	11/03/26
1.6	Implementation	96 hrs	11/03/26	24/04/26
1.7	Testing & validation	39 hrs	24/04/26	08/05/26
1.8	Documentation & closing	24 hrs	08/05/26	18/05/26

Table 2. WBS

As well as the WBS, here you will find a Gantt diagram (**Figure 3. Gantt Diagram**) that will help better see the duration of each task:

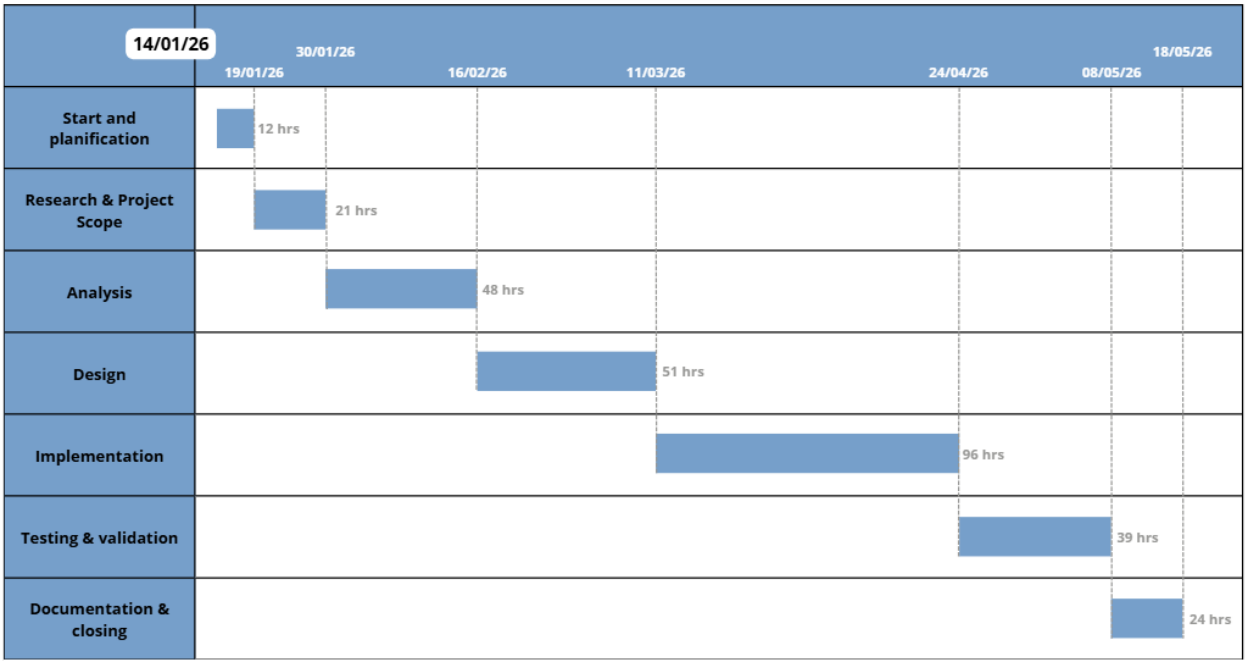

Figure 3. Gantt Diagram

Now, on **Table 3. Initial detailed WBS**, we will specify all subtasks inside each activity, as well as their durations:

WBS	Task Name	Work	Start	Finish
1	**Lunar Robot Coordination**	**300,5 hrs**	**Wed 14/01/26**	**Wed 20/05/26**
1.1	**Periodic meeting**	**9,5 hrs**	**Wed 14/01/26**	**Wed 20/05/26**
1.1.1	Periodic meeting 1	0,5 hrs	Wed 14/01/26	Wed 14/01/26
1.1.2	Periodic meeting 2	0,5 hrs	Wed 21/01/26	Wed 21/01/26
1.1.3	Periodic meeting 3	0,5 hrs	Wed 28/01/26	Wed 28/01/26
1.1.4	Periodic meeting 4	0,5 hrs	Wed 04/02/26	Wed 04/02/26
1.1.5	Periodic meeting 5	0,5 hrs	Wed 11/02/26	Wed 11/02/26
1.1.6	Periodic meeting 6	0,5 hrs	Wed 18/02/26	Wed 18/02/26
1.1.7	Periodic meeting 7	0,5 hrs	Wed 25/02/26	Wed 25/02/26
1.1.8	Periodic meeting 8	0,5 hrs	Wed 04/03/26	Wed 04/03/26
1.1.9	Periodic meeting 9	0,5 hrs	Wed 11/03/26	Wed 11/03/26
1.1.10	Periodic meeting 10	0,5 hrs	Wed 18/03/26	Wed 18/03/26
1.1.11	Periodic meeting 11	0,5 hrs	Wed 25/03/26	Wed 25/03/26
1.1.12	Periodic meeting 12	0,5 hrs	Wed 01/04/26	Wed 01/04/26
1.1.13	Periodic meeting 13	0,5 hrs	Wed 08/04/26	Wed 08/04/26
1.1.14	Periodic meeting 14	0,5 hrs	Wed 15/04/26	Wed 15/04/26

1.1.15	Periodic meeting 15	0,5 hrs	Wed 22/04/26	Wed 22/04/26
1.1.16	Periodic meeting 16	0,5 hrs	Wed 29/04/26	Wed 29/04/26
1.1.17	Periodic meeting 17	0,5 hrs	Wed 06/05/26	Wed 06/05/26
1.1.18	Periodic meeting 18	0,5 hrs	Wed 13/05/26	Wed 13/05/26
1.1.19	Periodic meeting 19	0,5 hrs	Wed 20/05/26	Wed 20/05/26
1.2	**Start and planification**	**12 hrs**	**Wed 14/01/26**	**Mon 19/01/26**
1.2.1	Problem definition	2 hrs	Wed 14/01/26	Wed 14/01/26
1.2.2	Find stakeholders, OBS & PBS	2 hrs	Wed 14/01/26	Wed 14/01/26
1.2.3	Create initial schedule (WBS)	4 hrs	Wed 14/01/26	Fri 16/01/26
1.2.4	Create initial budget	4 hrs	Fri 16/01/26	Mon 19/01/26
1.3	**Research & Project Scope**	**21 hrs**	**Mon 19/01/26**	**Fri 30/01/26**
1.3.1	Review of existing papers	17 hrs	Mon 19/01/26	Wed 28/01/26
1.3.2	Study of the current scope	4 hrs	Wed 28/01/26	Fri 30/01/26
1.4	**Analysis**	**48 hrs**	**Fri 30/01/26**	**Mon 16/02/26**
1.4.1	Analysis of client needs	8 hrs	Fri 30/01/26	Fri 30/01/26
1.4.2	**Requirement specification**	**40 hrs**	**Fri 30/01/26**	**Mon 16/02/26**
1.4.2.1	Define user requirements	10 hrs	Fri 30/01/26	Fri 06/02/26
1.4.2.2	Define system & functional requirements	20 hrs	Fri 06/02/26	Wed 11/02/26
1.4.2.3	Define non-functional requirements	10 hrs	Wed 11/02/26	Mon 16/02/26
1.5	**System design**	**51 hrs**	**Mon 16/02/26**	**Wed 11/03/26**
1.5.1	System architecture design	11 hrs	Mon 16/02/26	Mon 23/02/26
1.5.2	Communications & network design	20 hrs	Mon 23/02/26	Fri 27/02/26
1.5.3	Exploration logic design	11 hrs	Fri 27/02/26	Fri 06/03/26
1.5.4	User interface & visualization design	9 hrs	Fri 06/03/26	Wed 11/03/26
1.6	**Implementation**	**96 hrs**	**Wed 11/03/26**	**Fri 24/04/26**
1.6.1	Environment setup & configuration	8 hrs	Wed 11/03/26	Mon 16/03/26
1.6.2	Map processing module	17 hrs	Mon 23/03/26	Wed 01/04/26
1.6.3	Network & Node module	15 hrs	Mon 16/03/26	Mon 23/03/26
1.6.4	Exploration module	11 hrs	Fri 10/04/26	Wed 15/04/26
1.6.5	Visualization & UI module	15 hrs	Wed 01/04/26	Fri 10/04/26
1.6.6	Main simulation engine	30 hrs	Wed 15/04/26	Fri 24/04/26
1.7	**Testing and validation**	**39 hrs**	**Fri 24/04/26**	**Fri 08/05/26**
1.7.1	Unit testing	11 hrs	Fri 24/04/26	Wed 29/04/26
1.7.2	Integration testing	13 hrs	Wed 29/04/26	Fri 01/05/26
1.7.3	System testing	13 hrs	Mon 04/05/26	Wed 06/05/26
1.7.4	Code revision & clean-up	2 hrs	Wed 06/05/26	Fri 08/05/26
1.8	**Documentation & closing**	**24 hrs**	**Fri 08/05/26**	**Mon 18/05/26**
1.8.1	Create technical manual	11 hrs	Fri 08/05/26	Mon 11/05/26
1.8.2	Create user manual	5 hrs	Mon 11/05/26	Wed 13/05/26
1.8.3	Crate final budget	4 hrs	Wed 13/05/26	Fri 15/05/26
1.8.4	Conclusions, annexes & final risk report	4 hrs	Fri 15/05/26	Mon 18/05/26

Table 3. Initial detailed WBS

3.1.5. Risks

3.1.5.1. Risk management plan

This section will be detailed in the annex 9.3. Risk Management Plan.

3.1.5.2. Risk identification

The following risks were identified during the planning of this project:

1. **Not giving the client what they ask for.** The final product developed does not meet the needs, functional requirements, or initial expectations set by the client, resulting in a system that is not fit for purpose.
2. **Matplotlib Graphics Performance Bottlenecks.** When scaling the number of robots or loading a very large GIS map in main.py, updating the graphics may freeze the interface.
3. **Synchronization failures and UDP packet loss.** Due to the UDP protocol used, JSON packets or relay change commands may be lost or arrive out of order, desynchronizing the network state.
4. **Delays in the documentation phase.** Underestimating the mental effort and time required to write the report could jeopardize the submission deadline.
5. **Errors when processing GIS (Geographical Information System) elevation files (.dt2).** Problems when using *rasterio* and *pyproj* to convert GPS coordinates to UTM, or failures in handling null values, causing the simulation to start in an empty area or with errors.
6. **Total or partial loss of source code.** A physical failure in the disk, theft of the laptop, or accidental deletion of scripts without having a backup.
7. **Thread conflicts (*Tkinter* and main simulation).** Running the main function in the interface and then transferring control to the simulation can cause thread conflicts, leaving the window frozen.
8. **Unplanned scope creep.** During the validation phases, ideas for new functionalities may arise that exceed the time limit of the original final degree project.
9. **Logical failures in the routing algorithm (BFS).** The distance calculation logic could fail in extreme scenarios, causing operational robots to be declared as "isolated".
10. **Temporary incapacity of the lead developer.** If there is a leave of absence due to illness, accident or other personal problems for one or more weeks, the progress of the final degree project is stopped at 100%.

Now that the risks are identified, here you can see a table (**Table 4. Risk identification**) specifying an ID for each risk, a name, a brief description, and the category of each one.

ID	Risk Name	Brief description	Category
1	Not giving the client what they ask for	Misalignment between expectations and the product.	Project risk, Predictable
2	Matplotlib Graphics Performance Bottlenecks	Severe slowdown of the simulation.	Product risk, known risk
3	Synchronization failures and UDP packet loss	Loss of telemetry in the simulated network	Product risk, predictable
4	Delays in the documentation phase	Failure to meet deadlines in drafting.	Project risk, known risk
5	Errors when processing GIS elevation files (.dt2)	Critical failure in the map module.	Product risk, predictable
6	Total or partial loss of source code	Hardware failure or accidental deletion.	Project risk, unpredictable
7	Thread conflicts (Tkinter and main simulation)	UI delays at the start of execution	Product risk, predictable
8	Unplanned scope creep	Continuous addition of new requirements.	Project risk, predictable
9	Logical failures in the routing algorithm (BFS)	Erroneous disconnection of nodes.	Product risk, known risk
10	Temporary incapacity of the lead developer	Stoppage due to causes beyond our control	Project risk, unpredictable

Table 4. Risk identification

3.1.5.3. Risk register

The previously identified risks will be now detailed with ID, description, category, probability, impact, response and strategy of each one of the items.

Risk 1. Not giving the client what they ask for

Description	Misalignment between expectations and the product.		
Category	Project risk, Predictable		
Probability	Medium		
Impact	Budget	Imperceptible	**0,45**
	Planification	High	

	Scope	Critical	
	Quality	Critical	
Strategy	Mitigate the risk		
Response	Weekly meetings with the client		

Table 5. Risk 1

Risk 2. Matplotlib Graphics Performance Bottlenecks

Description	Severe slowdown of the simulation.		
Category	Product risk, known risk		
Probability	High		
Impact	Budget	Imperceptible	**0,39**
	Planification	Medium	
	Scope	Medium	
	Quality	High	
Strategy	Mitigate the risk		
Response	Optimizing the visual code and limiting the map resolution.		

Table 6. Risk 2

Risk 3. Synchronization failures and UDP packet loss

Description	Loss of telemetry in the simulated network		
Category	Product risk, predictable		
Probability	High		
Impact	Budget	Imperceptible	**0,63**
	Planification	Medium	
	Scope	Low	
	Quality	Critical	

Strategy	Mitigate the risk
Response	Implement a basic confirmation system or constantly forward telemetry on the main loop

Table 7. Risk 3

Risk 4. Delays in the documentation phase

Description	Failure to meet deadlines in drafting.		
Category	Project risk, known risk		
Probability	Medium		
Impact	Budget	Imperceptible	**0,45**
	Planification	Critical	
	Scope	Low	
	Quality	Medium	
Strategy	Mitigate the risk		
Response	Utilize project slack and add draft delivery milestones		

Table 8. Risk 4

Risk 5. Errors when processing GIS elevation files (.dt2)

Description	Critical failure in the map module.		
Category	Product risk, predictable		
Probability	Medium		
Impact	Budget	Imperceptible	**0,45**
	Planification	Medium	
	Scope	High	
	Quality	Critical	
Strategy	Accept the risk		

Response	Add try-except blocks with a default map.

Table 9. Risk 5

Risk 6. Total or partial loss of source code

Description	Hardware failure or accidental deletion.		
Category	Project risk, unpredictable		
Probability	Very low		
Impact	Budget	Low	**0,09**
	Planification	Critical	
	Scope	Critical	
	Quality	Medium	
Strategy	Eliminate the risk		
Response	Use version control (GitHub) and upload code to the cloud periodically.		

Table 10. Risk 6

Risk 7. Thread conflicts (Tkinter and main simulation)

Description	UI delays at the start of execution		
Category	Product risk, predictable		
Probability	Medium		
Impact	Budget	Imperceptible	**0,28**
	Planification	High	
	Scope	Medium	
	Quality	High	
Strategy	Mitigate the risk		

Response	Isolate the process from the simulation process interface, ensuring that the window is properly hidden before launching Matplotlib.

Table 11. Risk 7

Risk 8. Unplanned scope creep

Description	Continuous addition of new requirements.		
Category	Project risk, predictable		
Probability	High		
Impact	Budget	Imperceptible	**0,63**
	Planification	Critical	
	Scope	Critical	
	Quality	Medium	
Strategy	Eliminate the risk		
Response	Move any new ideas to Chapter 8 ("Expansions") as future work.		

Table 12. Risk 8

Risk 9. Logical failures in the routing algorithm (BFS)

Description	Erroneous disconnection of nodes.		
Category	Product risk, known risk		
Probability	Medium		
Impact	Budget	Imperceptible	**0,45**
	Planification	Medium	
	Scope	Low	
	Quality	Critical	
Strategy	Mitigate the risk		

Response	Design specific test cases for borderline coverage situations.

Table 13. Risk 9

Risk 10. Temporary incapacity of the lead developer

Description	Stoppage due to causes beyond our control		
Category	Project risk, unpredictable		
Probability	Very low		
Impact	Budget	Imperceptible	**0,09**
	Planification	Critical	
	Scope	Imperceptible	
	Quality	Imperceptible	
Strategy	Accept the risk		
Response	Utilize project slack or ask for an extension.		

Table 14. Risk 10

3.1.6. Initial budget

The budget at the initial state of the project will be detailed in this section. First, the cost budget, including the cost of materials and employees involved, will be explained.

3.1.6.1. Cost budget

The cost budget will be obtained from breaking down the hours of work of each professional role, in a way that each of the previously explained activities will have a specific cost.

3.1.6.1.1. Company definition

Even though this final degree project is made individually, we will simulate a small company in order to develop the budget for this development. Starting from the previously defined roles (project manager, system analyst, software architect, developer and tester), we have the following salaries and their price/hour (seen on **Table 15. Cost per role**).

These prices/hours were obtained after taking into account the amount needed for each employees' social security, their amount of productivity, and their direct and indirect costs.

Professional Rol	Annual Gross Salary	Price/Hour (Without benefits)
Project Manager	47.000,00 €	51,01 €
System Analyst	31.500,00 €	34,13 €
Software architect	38.000,00 €	38,89 €
Developer	35.000,00 €	35,39 €
Tester	30.000,00 €	34,73 €

Table 15. Cost per role

These prices will later be used to calculate the cost of developing each activity in the planification.

3.1.6.1.2. Services and licenses

In the development of this project, a series of equipment and licenses would be needed in our company. The services and their prices can be seen on **Table 16. Services budget**:

Services		
Services	**Cost month**	**Cost year**
Cleaning	300,00 €	3.600,00 €
Maintenance	300,00 €	3.600,00 €
Electricity consumption	1.000,00 €	12.000,00 €
Water consumption	100,00 €	1.200,00 €
Office rent	1.500,00 €	18.000,00 €
Network and communications service	100,00 €	1.200,00 €
Administrative costs	300,00 €	3.600,00 €
TOTAL	**3.600,00 €**	**43.200,00 €**

Table 16. Services budget

We have the cost of services per month, and our project lasts for roughly 5 months. So, the budget for services corresponding to our project will be (3.600€/month) * 5 months = **18.000€.**

The licenses and equipment needed by the team can be seen on detail in **Table 17. Equipment & licenses budgetTable 16. Services budget**:

Costs of the means of production						
Equipment/licence	**Units**	**Price**	**Total cost**	**Cost year**	**Type**	**Term**
Development equipment	5	1.200,00 €	6.000,00 €	1.500,00 €	Amortization	4
Development license	4	200,00 €	800,00 €	9.600,00 €	Monthly subscription	
Office365 license	2	7,00 €	14,00 €	168,00 €	Monthly subscription	
Github pro license	5	4,00 €	20,00 €	240,00 €	Monthly subscription	
MatLab Startup license	1	3.885,00 €	3.885,00 €	3.885,00 €	Yearly subscription	
Google AI Pro license	1	21,99 €	21,99 €	263,88 €	Monthly subscription	
TOTAL				**15.656,88 €**		

Table 17. Equipment & licenses budget

As we can see, we will need a series of licenses to make the development of the project possible:

- **Development license**. This license is needed for a professional Integrated Development Environment (IDE) to efficiently write, test, and debug the Python code.
- **Office365 license**. Required to handle all project management, administration, and documentation tasks throughout the lifecycle of the thesis.
- **GitHub Pro license.** Crucial for robust version control, secure cloud backups, and safeguarding the intellectual property of the project through private repositories.
- **MATLAB Startup license.** Provides a specialized environment to perform initial calculations for the swarm coordination, trajectory tracking, and data analysis of the terrain metrics. This license includes over 90 toolboxes, including the ones needed for this development.
- **Google AI Pro license.** Accelerating the research and development phases through advanced artificial intelligence assistance. The Pro tier provides enterprise-grade security and data privacy, ensuring that the proprietary simulation code and unpublished data remain confidential and are protected from public exposure.

We have the cost of licenses per year, and our project lasts for roughly 5 months. So, the budget for licenses corresponding to our project will be ((15.656,88€/year)/12 months) * 5 months = **6.523,70 €.**

3.1.6.1.3. Development

To obtain the budget of the development of the project, we will first calculate the cost of developing each one of the activities mentioned in the planification, with the hours spent by the different roles on that activity.

1. "Periodic meeting" costs

The project manager spent 9,5 hours throughout the development in periodic meetings. The price/hour (without benefit) for this role, is 51,01€ as we explained earlier.

This way we can say that the budget for the periodic meetings should be 51,01 * 9,5 = **484,59 €**.

2. "Start & Planification" costs

Start and planification							
I1	**I2**	**Description**	**Amount**	**Units**	**Price**	**Subtotal**	**Total**
01		**Problem definition**					**85,14 €**
	01	Project Manager	1	hours	51,01 €	51,01 €	
	02	Analyst	1	hours	34,13 €	34,13 €	
02		**Find stakeholders, OBS & PBS**					**102,02 €**
	01	Project Manager	2	hours	51,01 €	102,02 €	
03		**Create initial schedule (WBS)**					**204,05 €**
	01	Project Manager	4	hours	51,01 €	204,05 €	
04		**Create initial budget**					**204,05 €**
	01	Project Manager	4	hours	51,01 €	204,05 €	
					TOTAL A1:		**595,25 €**

Table 18. Start & planification budget

3. "Research and scope of the project" costs

Research and project scope							
I1	**I2**	**Description**	**Amount**	**Units**	**Price**	**Subtotal**	**Total**
01		**Review of existing papers**					**580,20 €**
	01	Analyst	17	hours	34,13 €	580,20 €	
02		**Study of the current scope**					**170,28 €**
	01	Project Manager	2	hours	51,01 €	102,02 €	
	02	Analyst	2	hours	34,13 €	68,26 €	
					TOTAL M2:		**750,48 €**

Table 19. Research & project scope budget

4. "Analysis" costs

Analysis							
I1	I2	Description	Amount	Units	Price	Subtotal	Total
01		**Analysis of client needs**					340,56 €
	01	Project Manager	4	hours	51,01 €	204,05 €	
	02	Analyst	4	hours	34,13 €	136,52 €	
02		**Define user requirements**					341,30 €
	01	Analyst	10	hours	34,13 €	341,30 €	
03		**Define system & functional requirements**					730,20 €
	01	Analyst	10	hours	34,13 €	341,30 €	
	02	Software Architect	10	hours	38,89 €	388,90 €	
04		**Define non-functional requirements**					388,90 €
	01	Software Architect	10	hours	38,89 €	388,90 €	
					TOTAL A3:		1.800,96 €

Table 20. Analysis budget

5. “Design” costs

System Design							
I1	I2	Description	Amount	Units	Price	Subtotal	Total
01		**System architecture design**					427,79 €
	01	Software Architect	11	hours	38,89 €	427,79 €	
02		**Communication & network design**					742,82 €
	01	Software Architect	10	hours	38,89 €	388,90 €	
	02	Developer	10	hours	35,39 €	353,92 €	
03		**Exploration logic design**					427,79 €
	01	Software Architect	11	hours	38,89 €	427,79 €	
04		**User interface & visualization design**					318,53 €
	01	Developer	9	hours	35,39 €	318,53 €	
					TOTAL A4:		1.916,93 €

Table 21. System design budget

6. “Implementation” costs

Implementation							
I1	I2	Description	Amount	Units	Price	Subtotal	Total
01		Environment setup & configuration					283,14 €
	01	Developer	8	hours	35,39 €	283,14 €	
02		Map processing module					601,66 €
	01	Developer	17	hours	35,39 €	601,66 €	
03		Network & Node module					530,88 €
	01	Developer	15	hours	35,39 €	530,88 €	
04		Exploration Module					389,31 €
	01	Developer	11	hours	35,39 €	389,31 €	
05		Visualization& UI Module					530,88 €
	01	Developer	15	hours	35,39 €	530,88 €	
06		Main Simulation Engine					1.114,23 €
	01	Software Architect	15	hours	38,89 €	583,35 €	
	02	Developer	15	hours	35,39 €	530,88 €	
					TOTAL A5:		3.450,09 €

Table 22. Implementation budget

7. "Testing & validation" costs

Testing & Validation							
I1	I2	Description	Amount	Units	Price	Subtotal	Total
01		Unit testing					382,06 €
	01	Tester	11	hours	34,73 €	382,06 €	
02		Integration testing					478,55 €
	01	Software Architect	6,5	hours	38,89 €	252,79 €	
	02	Tester	6,5	hours	34,73 €	225,76 €	
03		System testing					557,33 €
	01	Project Manager	6,5	hours	51,01 €	331,57 €	
	02	Tester	6,5	hours	34,73 €	225,76 €	
04		Code revision & clean-up					70,78 €
	01	Developer	2	hours	35,39 €	70,78 €	
					TOTAL A6:		1.488,72 €

Table 23. Testing & Validation budget

8. "Documentation & closing" costs

Documentation & Closing							
I1	I2	Description	Amount	Units	Price	Subtotal	Total
01		**Create technical manual**					408,55 €
	01	Software Architect	5,5	hours	38,89 €	213,90 €	
	02	Developer	5,5	hours	35,39 €	194,66 €	
02		**Create user manual**					170,65 €
	01	Analyst	5	hours	34,13 €	170,65 €	
03		**Create final budget**					204,05 €
	01	Project Manager	4	hours	51,01 €	204,05 €	
04		**Conclusions, anexes & final risk report**					204,05 €
	01	Project Manager	4	hours	51,01 €	204,05 €	
					TOTAL A7:		987,29 €

Table 24. Documentation & Closing budget

After calculating the budget needed for each activity in the development, we have found that the budget needed for the whole development (not taking into account licenses and equipment) will be **11.474,32€,** as can be seen on **Table 25. Development budget**.

Development budget	
Meetings	484,59 €
Start & Planification	595,25 €
Research & Project scope	750,48 €
Analysis	1.800,96 €
Design	1.916,93 €
Implementation	3.450,09 €
Testing & Validation	1.488,72 €
Documentation & Closing	987,29 €
TOTAL:	**11.474,32 €**

Table 25. Development budget

3.1.6.1.4. Final cost budget specification

The final cost budget will be composed of the budget for the services, the licenses budget and the budget destined to the development.

Following the risk register we specified earlier, we decided to add a contingency reserve in case any problems arise, this will be a 10% of the subtotal cost.

The initial cost budget can be seen on **Table 26. Initial Cost Budget**

Cost Budget	
Services Budget	18.000,00 €
License Budget	6.523,70 €
Development Budget	11.474,32 €
SubTotal	**35.998,02 €**
Contingency Reserve (10%)	3.599,80 €
TOTAL:	**39.597,82 €**

Table 26. Initial Cost Budget

The final cost budget is how much it will cost us as a company to complete this product, **39.597,82€**.

3.1.6.2. Client budget

The client budget will be calculated using the cost budget (what we need to complete the product), the profit margin we want (25%) and the VAT (Value Added Task) we need to add by law to any product (21% in Spain).

The initial client budget can be seen on **Table 27. Initial Client Budget**.

Client Budget	
Project ExecutionCost	39.597,82 €
Profit Margin (25%)	9.899,46 €
SubTotal	**49.497,28 €**
Taxes (21%)	10.394,43 €
TOTAL:	**59.891,71 €**

Table 27. Initial Client Budget

The final client budget is how much we would ask from the client, **59.891,71€**.

3.2. Project closure

3.2.1. Project incident log

During the development of this project, a few technical issues that we didn't foresee came up. These incidents are documented here to understand how they affected the initial planned schedule.

The most notable incidents were:

INC-01. Coordinate System Doesn't Match. During the development of the topographical module, a critical inconsistency was discovered. The imported maps give the coordinates on geographical degrees (based on latitude and longitude), while the robots' movements are calculated in the metric system.

Resolution. This was solved using the *pyproj* library, using their "Transformer" class, to convert the GPS coordinates (EPSG:4326) into the UTM metric system (EPSG:32628).

INC-02. Visualization Engine Full-Screen. The requirement to display the Matplotlib simulation in full-screen mode caused unexpected conflicts with the backend. Maximize commands failed or caused the UI to freeze after closing the initial configuration window.

Resolution. The issue was fixed by accessing the active figure manager (plt.get_current_fig_manager()) and explicitly passing the state('zoomed') parameter.

INC-03. Blocking due to the Synchronous Sockets. During the first version of the RobotNode class, the receiver method for messages was stopping the simulation waiting for the telemetry package to arrive before continuing.

Resolution. The architecture was refactored to set the UDP sockets to non-blocking mode (sock.setblocking(False)), and the receiver logic was turned into a background daemon using threads.

INC-04. Same area explored twice in Fixed Exploration Mode. After hitting a side of the exploration area and bouncing back, rovers would go over twice exploring the same path already explored before hitting the wall.

Resolution. A randomized angular noise factor (between -pi/6 and pi/6 radians) was introduced, this ensures continuous area coverage and forcing the rovers to follow a new trajectory.

INC-05. Matplotlib Slowing Down. In earlier version, the plot, markers and lines were drawn continuously at every step.

Resolution. We optimized the visualization.py logic. Instead of redrawing the whole canvas at every step, we only delete and redraw the parts that changed (the position of the rovers, explored area and network lines).

3.2.2. Final risk report

Comparing the execution of the project with the initial Risk Register (3.1.5.3. Risk register) the following conclusions can be seen regarding what threads materialized and which didn't (seen on **Table 28. Final risk report**):

Risk ID	Status
Risk 1	Didn't materialize
Risk 2	Didn't materialize
Risk 3	Didn't materialize
Risk 4	Didn't materialize
Risk 5	Materialized
Risk 6	Didn't materialize
Risk 7	Materialized
Risk 8	Didn't materialize
Risk 9	Didn't materialize
Risk 10	Didn't materialize

Table 28. Final risk report

Risk 5 (Errors when processing GIS elevation files). This risk **materialized** in the form of the coordinate problem explained in INC-01. However, the contingency time was used to implement the pyproj transformation without stopping the entire project.

Risk 7 (Thread conflicts - UI and main simulation). This risk **materialized** in the incidents explained in INC-02 and INC-03. The UI experienced delays, but the mitigation strategy (isolating the process and hiding the window properly) was applied. Threads also caused conflicts, but were solved making the processes non-blocking as explained.

Overall, the initial risk identification was accurate, and the mitigation strategies effective.

3.2.3. Final cost budget

To reflect the previously explained incidents encountered, the planification suffered a few deviations from what was originally stated on 3.1.4. Initial plan and WBS (Work Breakdown Structure), the following adjustments were made:

- Task 1.6.2. (Map processing module implementation) was increased from 17 hours to 22 hours, for the problems explained regarding the coordinate match.
- Task 1.6.5. (Visualization & UI Module) was increased from 15 hours to 20. More time was spent debugging and fixing the issues with the threads and Matplotlib conflicts.
- Tasks 1.4.1-1.4.2.3 (All tasks in the Analysis phase) went up by 2 hours. This was because more time was spent on displaying the requirements in a tree shape, better in technical documentation.
- Task 1.8.4 (Conclusions, annexes & final risk report) went from 4 hours to 10 hours. The 8.2. Expansions section took more time than expected, as it involved further investigation not made during the initial phase dedicated to it.
- A vital task was omitted on the initial schedule for the project, and it was now added as task **1.8.5. Executable creation**, and was assigned 4 hours to be completed by the Developer.

This is the final project plan, which went from 300,5 hours in the initial plan, to 328,5 hours. On **Table 29. Final detailed WBS** the duration and execution dates of each activity can be seen.

WBS	Task Name	Work	Start	Finish
1	**Lunar Robot Coordination**	**328,5 hrs**	**Wed 14/01/26**	**Mon 01/06/26**
1.1	**Periodic meeting**	**9,5 hrs**	**Wed 14/01/26**	**Wed 20/05/26**
1.1.1	Periodic meeting 1	0,5 hrs	Wed 14/01/26	Wed 14/01/26
1.1.2	Periodic meeting 2	0,5 hrs	Wed 21/01/26	Wed 21/01/26
1.1.3	Periodic meeting 3	0,5 hrs	Wed 28/01/26	Wed 28/01/26
1.1.4	Periodic meeting 4	0,5 hrs	Wed 04/02/26	Wed 04/02/26
1.1.5	Periodic meeting 5	0,5 hrs	Wed 11/02/26	Wed 11/02/26
1.1.6	Periodic meeting 6	0,5 hrs	Wed 18/02/26	Wed 18/02/26
1.1.7	Periodic meeting 7	0,5 hrs	Wed 25/02/26	Wed 25/02/26
1.1.8	Periodic meeting 8	0,5 hrs	Wed 04/03/26	Wed 04/03/26
1.1.9	Periodic meeting 9	0,5 hrs	Wed 11/03/26	Wed 11/03/26
1.1.10	Periodic meeting 10	0,5 hrs	Wed 18/03/26	Wed 18/03/26
1.1.11	Periodic meeting 11	0,5 hrs	Wed 25/03/26	Wed 25/03/26
1.1.12	Periodic meeting 12	0,5 hrs	Wed 01/04/26	Wed 01/04/26
1.1.13	Periodic meeting 13	0,5 hrs	Wed 08/04/26	Wed 08/04/26

1.1.14	Periodic meeting 14	0,5 hrs	Wed 15/04/26	Wed 15/04/26
1.1.15	Periodic meeting 15	0,5 hrs	Wed 22/04/26	Wed 22/04/26
1.1.16	Periodic meeting 16	0,5 hrs	Wed 29/04/26	Wed 29/04/26
1.1.17	Periodic meeting 17	0,5 hrs	Wed 06/05/26	Wed 06/05/26
1.1.18	Periodic meeting 18	0,5 hrs	Wed 13/05/26	Wed 13/05/26
1.1.19	Periodic meeting 19	0,5 hrs	Wed 20/05/26	Wed 20/05/26
1.2	**Start and planification**	**12 hrs**	**Wed 14/01/26**	**Mon 19/01/26**
1.2.1	Problem definition	2 hrs	Wed 14/01/26	Wed 14/01/26
1.2.2	Find stakeholders, OBS & PBS	2 hrs	Wed 14/01/26	Wed 14/01/26
1.2.3	Create initial schedule (WBS)	4 hrs	Wed 14/01/26	Fri 16/01/26
1.2.4	Create initial budget	4 hrs	Fri 16/01/26	Mon 19/01/26
1.3	**Research & Project Scope**	**21 hrs**	**Mon 19/01/26**	**Fri 30/01/26**
1.3.1	Review of existing papers	17 hrs	Mon 19/01/26	Wed 28/01/26
1.3.2	Study of the current scope	4 hrs	Wed 28/01/26	Fri 30/01/26
1.4	**Analysis**	**56 hrs**	**Fri 30/01/26**	**Fri 20/02/26**
1.4.1	Analysis of client needs	10 hrs	Fri 30/01/26	Mon 02/02/26
1.4.2	**Requirement specification**	**46 hrs**	**Mon 02/02/26**	**Fri 20/02/26**
1.4.2.1	Define user requirements	12 hrs	Mon 02/02/26	Fri 06/02/26
1.4.2.2	Define system & functional requirements	22 hrs	Fri 06/02/26	Fri 13/02/26
1.4.2.3	Define non-functional requirements	12 hrs	Fri 13/02/26	Fri 20/02/26
1.5	**System design**	**51 hrs**	**Fri 20/02/26**	**Fri 13/03/26**
1.5.1	System architecture design	11 hrs	Fri 20/02/26	Wed 25/02/26
1.5.2	Communications & network design	20 hrs	Wed 25/02/26	Wed 04/03/26
1.5.3	Exploration logic design	11 hrs	Wed 04/03/26	Mon 09/03/26
1.5.4	User interface & visualization design	9 hrs	Mon 09/03/26	Fri 13/03/26
1.6	**Implementation**	**106 hrs**	**Fri 13/03/26**	**Fri 01/05/26**
1.6.1	Environment setup & configuration	8 hrs	Fri 13/03/26	Wed 18/03/26
1.6.2	Map processing module	22 hrs	Fri 27/03/26	Wed 08/04/26
1.6.3	Network & Node module	15 hrs	Wed 18/03/26	Fri 27/03/26
1.6.4	Exploration module	11 hrs	Fri 17/04/26	Fri 24/04/26
1.6.5	Visualization & UI module	20 hrs	Wed 08/04/26	Fri 17/04/26
1.6.6	Main simulation engine	30 hrs	Fri 24/04/26	Fri 01/05/26
1.7	**Testing and validation**	**39 hrs**	**Fri 01/05/26**	**Fri 15/05/26**
1.7.1	Unit testing	11 hrs	Fri 01/05/26	Fri 08/05/26
1.7.2	Integration testing	13 hrs	Fri 08/05/26	Mon 11/05/26
1.7.3	System testing	13 hrs	Mon 11/05/26	Fri 15/05/26
1.7.4	Code revision & clean-up	2 hrs	Fri 15/05/26	Fri 15/05/26
1.8	**Documentation & closing**	**34 hrs**	**Fri 15/05/26**	**Mon 01/06/26**
1.8.1	Create technical manual	11 hrs	Fri 15/05/26	Wed 20/05/26
1.8.2	Create user manual	5 hrs	Wed 20/05/26	Fri 22/05/26
1.8.3	Crate final budget	4 hrs	Fri 22/05/26	Mon 25/05/26
1.8.4	Conclusions, annexes & final risk report	10 hrs	Mon 25/05/26	Fri 29/05/26

1.8.5	Executable creation	4 hrs	Fri 29/05/26	Fri 29/05/26

Table 29. Final detailed WBS

With the final schedule finished, we can move on to recalculate the project budget again. The only things that will change will be the development costs as the hours of work increase, but data like employees' salaries and services/licenses costs will remain the same as specified in 3.1.6.1. Cost budget.

1. The "Periodic meeting" cost stays the same as specified in the initial budget **484,59€.**
2. The "Start and Planification" cost stays as specified in the initial budget **595,25€.**
3. The "Research and Project Scope" cost stays the same as specified in the initial budget **750,48€.**
4. The "Analysis" cost has increased from 1800,96€ to **2105,16€**.

Analysis							
I1	**I2**	**Description**	**Amount**	**Units**	**Price**	**Subtotal**	**Total**
01		**Analysis of client needs**					**425,70 €**
	01	Project Manager	5	hours	51,01 €	255,06 €	
	02	Analyst	5	hours	34,13 €	170,65 €	
02		**Define user requirements**					**409,55 €**
	01	Analyst	12	hours	34,13 €	409,55 €	
03		**Define system & functional requirements**					**803,22 €**
	01	Analyst	11	hours	34,13 €	375,43 €	
	02	Software Architect	11	hours	38,89 €	427,79 €	
04		**Define non-functional requirements**					**466,68 €**
	01	Software Architect	12	hours	38,89 €	466,68 €	
					TOTAL A3:		**2.105,16 €**

Table 30. Analysis final budget

5. The "System Design" cost stays as specified in the initial budget **1916,93€**.
6. The "Implementation" cost has increased from 3450,09€ to **3804,01€**.

Implementation							
I1	I2	Description	Amount	Units	Price	Subtotal	Total
01		**Environment setup & configuration**					283,14 €
	01	Developer	8	hours	35,39 €	283,14 €	
02		**Map processing module**					778,62 €
	01	Developer	22	hours	35,39 €	778,62 €	
03		**Network & Node module**					530,88 €
	01	Developer	15	hours	35,39 €	530,88 €	
04		**Exploration Module**					389,31 €
	01	Developer	11	hours	35,39 €	389,31 €	
05		**Visualization& UI Module**					707,84 €
	01	Developer	20	hours	35,39 €	707,84 €	
06		**Main Simulation Engine**					1.114,23 €
	01	Software Architect	15	hours	38,89 €	583,35 €	
	02	Developer	15	hours	35,39 €	530,88 €	
					TOTAL A5:		3.804,01 €

Table 31. Implementation final budget

7. The “Testing & Validation” cost stays as specified in the initial budget **1488,72€**.
8. The “Documentation & Closing” cost has increased from 987,29€ to **1128,86€**.

Documentation & Closing							
I1	I2	Description	Amount	Units	Price	Subtotal	Total
01		**Create technical manual**					408,55 €
	01	Software Architect	5,5	hours	38,89 €	213,90 €	
	02	Developer	5,5	hours	35,39 €	194,66 €	
02		**Create user manual**					170,65 €
	01	Analyst	5	hours	34,13 €	170,65 €	
03		**Create final budget**					204,05 €
	01	Project Manager	4	hours	51,01 €	204,05 €	
04		**Conclusions, anexes & final risk report**					204,05 €
	01	Project Manager	4	hours	51,01 €	204,05 €	
05		**Executable creation**					141,57 €
	01	Developer	4	hours	35,39 €	141,57 €	
					TOTAL A7:		1.128,86 €

Table 32. Documentation and closing final budget

After calculating the budget needed for each activity in the development, the budget needed for the whole development (not taking into account licenses and equipment) will be **12.274,01€**. The final development budget can be seen on **Table 33. Final development budget**.

Development budget	
Meetings	484,59 €
Start & Planification	595,25 €
Research & Project scope	750,48 €
Analysis	2.105,16 €
Design	1.916,93 €
Implementation	3.804,01 €
Testing & Validation	1.488,72 €
Documentation & Closing	1.128,86 €
TOTAL:	**12.274,01 €**

Table 33. Final development budget

Including the licenses and services (unchanged from Services and licenses section), we get that the cost budget for the project was **40.477,48€**. This is detailed on **Table 34. Final cost budget**.

Cost Budget	
Services Budget	18.000,00 €
License Budget	6.523,70 €
Development Budget	12.274,01 €
SubTotal	**36.797,71 €**
Contingency Reserve (10%)	3.679,77 €
TOTAL:	**40.477,48 €**

Table 34. Final cost budget

After applying our profit margin (25%) and taxes, we get that the client budget for the whole project will be **61.222,18€**. This is detailed on **Table 35. Final client budget**.

Client Budget	
Project ExecutionCost	40.477,48 €
Profit Margin (25%)	10.119,37 €
SubTotal	**50.596,85 €**
Taxes (21%)	10.625,34 €
TOTAL:	**61.222,18 €**

Table 35. Final client budget

3.2.4. Lessons learned report

The development of this project has given me valuable information that I will be able to apply into future developments.

1. **UI Framework Choice**. Matplotlib is a very powerful tool in terms of the simulation, but can be difficult to manage in terms of graphical user interfaces. Future projects might be better if they had the UI completely separated from the simulation, for example implementing it as a web dashboard.
2. **Standardization of Data Types**. Data mismatches, like the one seen on INC-01, should be found in during the development. For future projects, data relationships should be established during the Architecture Design phase.
3. **Risk Management**. As seen on 3.2.1. Project incident log and 3.2.2. Final risk report, the most time-consuming bugs were already identified in the initial Risk Register. Having anticipated these issues prevented panic and allowed for structured problem solving.
4. **Initial investigation**. For me, lunar exploration and geospatial data were a completely new field. I also think this is such a big field, that it is impossible to investigate everything, but I do wish I invested more time before starting the project to read research papers to better understand the scope of the problem.

Chapter 4. System requirements

4.1. Analysis

4.1.1. Geographic Data Processing Libraries

The system requires a library capable of parsing Digital Terrain Elevation Data (.dt2 files), extracting the elevation matrix, and handling Coordinate Reference System (CRS) transformations. The following alternatives were evaluated:

GDAL (Geospatial Data Abstraction Library)

GDAL is the industry standard for reading and manipulating raster geospatial data formats. It is extremely powerful, but its Python library (*osgeo.gdal*) is just wrapped around C/C++ code. This makes data extraction very complex and can cause installation issues on different operating systems.

Custom Parser in Pure Python

Another option would be developing our own parser to read the .dt2 byte by byte. This would eliminate third-party dependencies, but it would be too difficult and over-engineer the solution, when other libraries exist that do just what we need.

Rasterio

Rasterio is an open-source library built on top of GDAL, specifically designed to make geospatial data processing intuitive in Python. It integrates easily with *numpy*, and allows to handle missing data (nodata) natively.

Selected option

Rasterio was selected as the optimal solution, combined with the *pyproj* library for coordinate transformations. It allows to efficiently extract the elevation arrays and directly inject them into the simulation's topological grid.

4.1.2. Network Communication Protocols and Libraries

To simulate the exchange of telemetry and routing commands between the rovers, the system required a lightweight and asynchronous networking solution. The communication model needed to reflect the reality of radio transmissions in harsh environments, where

connections are often stateless and susceptible to packet loss. The following alternatives were evaluated:

TCP-based Protocols (WebSockets or HTTP REST)

These protocols ensure reliable, ordered, and error-checked delivery of a stream of data between applications. In a dynamic swarm where nodes are constantly moving, maintaining persistent TCP connections can introduce blocking behaviors.

Message Brokers (MQTT)

Publish-Subscribe (Pub/Sub) messaging protocols are the industry standard for IoT and distributed systems. Protocols like MQTT (Message Queuing Telemetry Transport) typically rely on a centralized server that routes all messages, defeating the purpose of simulating a decentralized peer-to-peer routing algorithm where robots can act as relays.

Native UDP Sockets (*socket* library)

User Datagram Protocol (UDP) is a connectionless, stateless communications protocol. Python's built-in socket library allows for low-level access to network interfaces. UDP allows nodes to "fire and forget" telemetry packets, this mimics real radios, where if a packet is lost due to distance, the simulation must handle it.

Selected option

Native UDP Sockets were selected as the communication protocol. By implementing non-blocking UDP receiver loops on separate threads, the simulation achieves a realistic, decentralized, and lightweight peer-to-peer network model without relying on external servers or heavy dependencies.

4.1.3. Graphical User Interface (GUI) Frameworks

The simulation required an initial configuration interface to capture the user's parameters (number of rovers, coordinates, area size, and exploration mode) or to handle the uploading of .json configuration files before launching the main graphical engine. The following alternatives were evaluated to develop this setup window:

Web-Based Interface (Flask/FastAPI + HTML/JS)

Building a local web server to host a modern, browser-based dashboard. While web interfaces offer modern aesthetics, introducing a client-server architecture for a local, standalone simulation introduces unnecessary difficulties and conflicts with the non-functional requirement NF-1of delivering a simple executable.

PyQt / PySide (Qt for Python)

A set of Python bindings for the Qt application framework, widely used for professional desktop applications. It is very powerful and provides advanced design tools, but it is too big of a dependency and would increase the final .exe file size too much.

Tkinter

The standard GUI library for Python, natively integrated into the Python Standard Library. t is extremely lightweight, fast, and provides all the necessary widgets to complete the configuration requirements. Furthermore, its native inclusion in Python guarantees seamless compatibility with packaging tools like *PyInstaller*.

Selected option

Tkinter was selected as the UI framework. It perfectly aligns with the project's scope: providing a functional, lightweight, and robust data-entry window that immediately transfers control to the matplotlib engine upon validation, keeping the final executable as lean and portable as possible.

4.1.4. 2D Rendering and Visualization Engines

The core requirement for the simulation's visual component was to continuously render the dynamic state of the robotic swarm (positions and network links) overlaid on the map topography. The following alternatives were evaluated:

Pygame (or similar2D game engines)

Pygame is a highly popular set of Python modules designed for writing video games, offering excellent frame rates. Game engines are built around pixel coordinates rather than scientific or geographic coordinate systems (UTM/meters), this causes it to be highly inefficient for mapping elevation arrays.

PyQtGraph

A pure-python graphics and GUI library built on PyQt/PySide, specifically designed for fast, real-time scientific plotting. As established in the GUI analysis (4.1.3. Graphical User Interface (GUI) Frameworks) introducing Qt into the project would drastically bloat the executable size and violate the portability constraints.

Matplotlib

The industry-standard, comprehensive library for creating static, animated, and interactive scientific visualizations in Python. Matplotlib integrates flawlessly with *numpy* multi-dimensional arrays and provides functionalities perfect for rendering elevation matrices.

Selected option

Matplotlib was selected as the rendering engine. It has the perfect balance between scientific accuracy and dynamic visualization. While it does not provide the ultra-high frame rates, its native ability to plot geographic coordinate axes, render heatmaps, and dynamically draw the network routing topology makes it the ideal tool.

4.1.5. Network Routing algorithm (MANET Protocols)

In swarm robotics, the rovers form a Mobile Ad Hoc Network (MANET), where nodes connect and disconnect dynamically. Before designing the routing logic, we need to evaluate the different types of algorithms available. The following routing paradigms were considered:

Optimized Link State Routing Protocol (OLSR)

OLSR is a proactive, table-driven routing protocol designed for MANETs. It is based on a central or global paradigm where each node maintains a complete view of the network topology by continuously exchanging control messages (HELLO messages for neighbor discovery and Topology Control messages for network dissemination).

It offers immediate route availability, deterministic routing and optimal path calculations [5]. For networks with a small number of rovers (less than 10), the control traffic overhead is very small. Additionally, it requires a monitoring node to reconstruct the full network graph and determine exact routes.

It requires every node to store and update the global state matrix constantly, which increases code complexity and memory usage for a pure simulation environment.

B.A.T.M.A.N. (Better Approach to Mobile Ad-hoc Networking)

BATMAN operates under a highly decentralized paradigm. Instead of maintaining a full map of the network, each node only knows which single neighboring node provides the best route to a given destination. The approach of the B.A.T.M.A.N algorithm is to divide the knowledge about the best end-to-end paths between nodes in the mesh to all participating nodes. [6]

It eliminates the need for calculating global routing tables, heavily reducing the computational load on individual nodes and minimizing routing loops.

In contrast to OLSR, it is difficult for a central Supervisor (SV) node to natively monitor and reconstruct the full network topology, as it only possesses partial routing information.

Breadth-First Search (BFS)

For this specific software simulation, a custom routing tree generation based on the Breadth-First Search (BFS) algorithm was developed within the Supervisor node.

Starting from the server node, neighboring robots within a predetermined distance threshold are explored. Additionally, a further restriction based on the nodes' battery level is introduced to prevent the use of low-power robots as repeaters. [7]

Selected option

The selected approach is a hybrid solution made specifically for the project's scope.

In our system, the Supervisor (SV) acts as the central intelligence (similar to OLSR's global view concept). On the other hand, each robot node holds a pointer to their relay, which will receive the messages (similar to how on BATMAN algorithms each node knows which single neighboring node provides the best route).

Finally, we created a BFS algorithm over the available distances between all nodes at every time step, and the SV calculates a routing tree. This BFS implementation successfully guarantees that all active explorer rovers are connected to the SV through the shortest path possible, without the massive protocol overhead of implementing full OLSR HELLO/TC messages, and providing much better topological visibility than a pure BATMAN implementation.

4.2. Functional requirements

4.2.1. System functions

SR-1. Introducing simulation parameters

SR-1.1: The graphical user interface shall allow the user to introduce the number of rovers that will be on the swarm.

- This input is mandatory if the user is going to introduce data manually.
- This input must be a number.
- This number must be between 3 and 7.
- This input must have 3 as a placeholder to guide the user.
- The default value for this input must be 3.

SR-1.2: The graphical user interface shall allow the user to introduce the relative coordinates (X and Y) of SV with respect to the center of the map.

- Both of these inputs are mandatory if the user is going to introduce data manually.
- Both of these inputs must be a number.
- These inputs must have 100 and -100 as a placeholder to guide the user.
- The default value for these inputs must be 0, the center of the map.

SR-1.3: The graphical user interface shall allow the user to introduce the initial size of the exploration area for the simulation.

- This input is mandatory if the user is going to introduce data manually.
- This input must be a number.
- This number must be positive.
- This input must be 120 as a placeholder to guide the user.
- The default value for this input must be 120.

SR-1.4: The graphical user interface shall allow the user to choose between “Free Exploration Mode” and “Fixed Exploration Mode”.

- This input is mandatory if the user is going to introduce data manually.
- The default value for this input must be “Fixed Exploration Mode”.

SR-1.5: The graphical user interface shall allow the user to introduce all simulation parameters through a configuration files, with .json extension.

- SR-1.5.1: The system must show the user a file browser where they can upload the file.
- SR-1.5.2: If the user has uploaded a file, manual inputs will be ignored.
    - The user must be warned before ignoring the manual inputs.

- SR-1.5.3: The configuration file must include a “num_rovers” field, with the number of rovers that will be on the swarm.
    - This field is mandatory.
    - This input must be a number.
    - This number must be between 3 and 7.
- SR-1.5.4: The configuration file must include a “offset_sv_x” field, with the relative coordinate (X) of SV with respect to the center of the map.
    - This field is mandatory.
    - This input must be a number.
- SR-1.5.5: The configuration file must include a “offset_sv_y” field, with the relative coordinate (Y) of SV with respect to the center of the map.
    - This field is mandatory.
    - This input must be a number.
- SR-1.5.6: The configuration file must include a “area_size” field, with initial size of the exploration area for the simulation.
    - This field is mandatory.
    - This input must be a number.
    - This number must be positive.
- SR-1.5.7: The configuration file must include a “exploration_mode” field, with the exploration mode desired.
    - This field is mandatory.
    - This input must be a number.
    - This number must be 1 or 2.
    - If the input is 1, it means the user chose “Free Exploration Mode”.
    - If the input is 2, it means the user chose “Fixed Exploration Mode”.

SR-1.6: The system must include a button to start the simulation, which executes the consequent checks.

- SR-1.6.1: If all checks pass, the simulation can begin.

SR-1.7: The system shall include information about the parameters introduced while the simulation is running.

SR-2. Communication between robots

SR-2.1: Each robot node must run an independent listening thread using non-blocking UDP sockets (socket.SOCK_DGRAM) to simulate asynchronous, real-time network communication.

SR-2.2: The communication protocol must encapsulate data in JSON packets containing, at a minimum, the following fields: origin, target, payload, and a read flag.

SR-2.3: A robot node loses connection with another when the distance between them is over 50 meters.

- SR-2.3.1: If it finds another robot node that can act as a relay, the simulation continues.
    - SR-2.3.1.1: The robot node that started acting as the new relay, stops moving as to not lose the connection.
    - SR-2.3.1.2: The robot node that changed its relay changes directions slightly, with hopes of covering different areas in the exploration.
- SR-2.3.2: If it is unable to find a relay (or chain of relays) that keeps it connected to SV, the simulation stops.
    - SR-2.3.2.1: The system must show the user a message with information about why the simulation finished, which robot lost connection.

SR-2.4: The system must enforce an energy constraint: an explorer robot can only act as a communication relay for other robots if its battery level is strictly greater than 20%.

SR-3. Terrain exploration functioning

SR-3.1: The system must provide the user with 2 different exploration modes.

- SR-3.1.1: The system must allow the user to perform the simulation with a “Free Exploration Mode”.
    - SR-3.1.1.1: In a simulation with “Free Exploration”, the system must allow the exploration area to expand dynamically as robots reach the edges of the currently mapped zone.
        - SR-3.1.1.1.1: The map must expand 50 meters on each side.
    - SR-3.1.1.2: In a simulation with “Free Exploration”, the system will conclude only when the simulation is canceled by the user or one of the rovers is unable to find a path to SV.
- SR-3.1.2: The system must allow the user to perform the simulation with a “Fixed Exploration Mode”.

- SR-3.1.2.1: In a simulation with “Fixed Exploration”, the system must stop the robots from exiting the area.
  - SR-3.1.2.1.1: The robot that touches a side of the exploration area, must bounce back inside with a random angle.
- SR-3.1.2.2: In a simulation with “Fixed Exploration”, the system will conclude the simulation when it is canceled by the user, one of the rovers loses connection to SV, or when all the area has been marked as explored.

SR-3.2: The exploration map must be represented as a two-dimensional grid matrix, where the cells can be either explored or not explored.

- SR-3.2.1: All cells in a radius of 20 meters around each rover must be marked as explored.
- SR-3.2.2: The system must calculate and show in real time the percentage of cells that have been explored from the exploration area.

SR-3.3: The system must include a Stop/Resume button, to stop the simulation at any given time.

4.2.2. Domain data model

Given the real-time simulation nature of this project, the system does not require a persistent relational database for long-term data storage. Instead, the Domain Data Model is implemented entirely in-memory using an Object-Oriented Programming (OOP) paradigm in Python. The data architecture revolves around a set of entities that represent the physical and logical components of the exploration swarm, as well as the information exchanged between them.

The UML Class Diagram below (**Figure 4. Domain Data Model**) illustrates these primary entities, their internal attributes, and the structural relationships that govern the simulation. The model is built with four fundamental entities:

- **RobotNode:** The central entity representing the physical rovers. It stores state data such as coordinates, velocity vectors, battery levels, and network port assignments. It distinguishes between the Supervisor (SV) and standard explorer nodes.
- **ExplorationMap:** The entity responsible for handling the spatial and environmental data. It translates real-world elevation data (derived from .dt2 files) into a grid-based matrix, tracking the discovered areas.

- **ExplorationStrategy:** An interface representing the movement logic (implementing the Strategy design pattern), containing the specific algorithms for free and fixed exploration modes.
- **Telemetry Data:** The structured JSON payloads that flow through the UDP sockets, acting as the dynamic data models for network communication, routing, and synchronization.

In addition to these classes, we also have others that contain vital functionalities that are needed in the simulation the utils.

Finally, to meet the requirements of this product, we developed a GUI.py, which contains all functions related to the user interface tool.

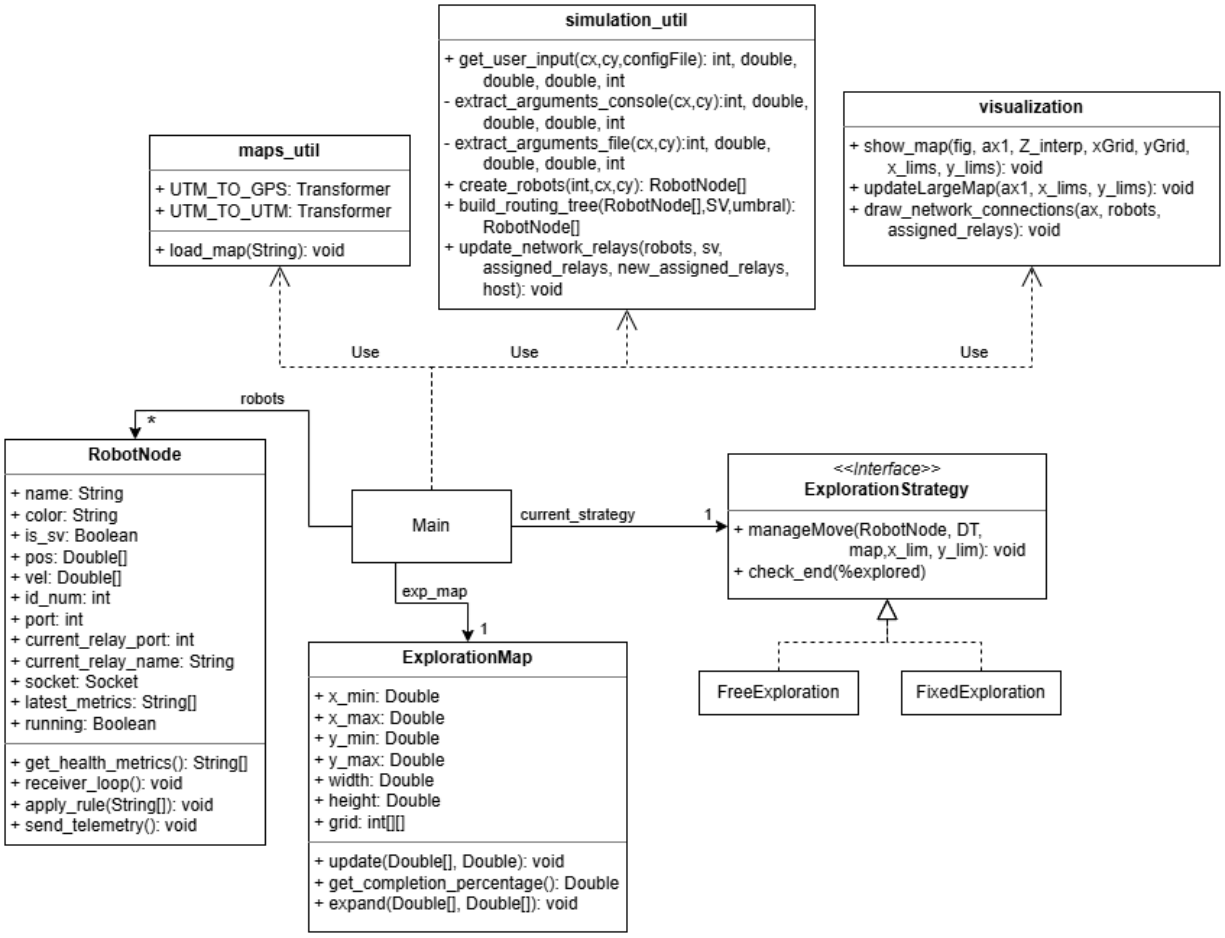

Figure 4. Domain Data Model

4.2.3. User interface

The User Interface (UI) for this project is designed as a single-window configuration screen. Its objective is to collect and validate the initial parameters required for the robot coordination simulation before launching the main Matplotlib graphical engine. The prototype (**Figure 5. UI Wireframe**) was designed prioritizing usability, offering the user two distinct methods to input data: manual entry or automated file upload.

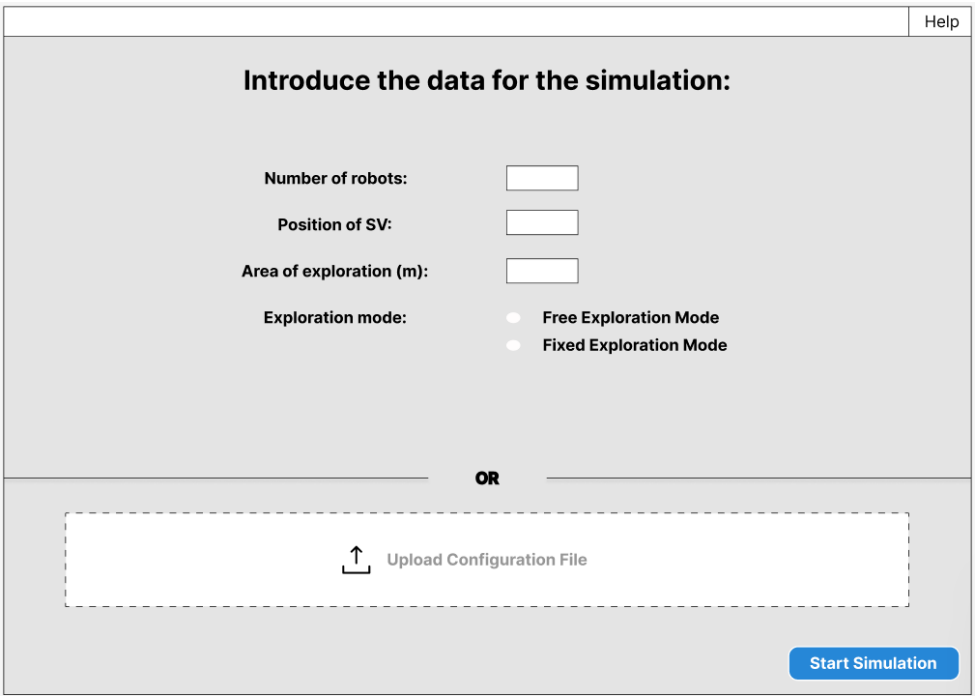

Figure 5. UI Wireframe

Based on the prototype, the interface is divided into the following logical components:

- **Manual Configuration Section.** This block allows the user to individually set the simulation variables. It includes a spinbox to define the number of robots in the swarm (restricted between 3 and 7), and text input fields to determine the initial X and Y coordinates of the Supervisor (SV) node, as well as the size of the exploration area. Placeholder text is used within the entry fields to guide the user on the expected data format.
- **Mode Selection.** A set of radio buttons that allows the user to choose the algorithmic behavior of the swarm, between Free Exploration Mode and Fixed Exploration Mode.
- **Automated Configuration (JSON Upload).** This section provides an alternative to the manual entry. It contains an "Upload JSON" button to load pre-configured scenarios. This drastically improves the user experience for repetitive testing phases.
- **Execution and Validation.** The "Start Simulation" button acts as the final trigger. Before closing the set-up window and launching the simulation engine, a validation process must occur (checking for empty fields, preventing negative area values, and prioritizing the JSON file if both input methods are filled), notifying the user via pop-up warnings if any errors are detected.

4.3. Non-functional requirements

4.3.1. Portability and deployment

NF-1: The final software must be compiled and packaged as a standalone executable file. End-users must be able to run the simulation without needing to install a Python interpreter, set up virtual environments, or manually download third-party dependencies.

NF-2: All necessary static assets, such as the digital elevation map files (.dt2 or .tif), must be correctly bundled or relatively linked to the executable to ensure seamless execution on any host machine.

4.3.2. Performance and efficiency

NF-3: The simulation engine must be capable of processing the physical movement, network logic, and graphical rendering to provide a fluid and continuous visual experience without freezing.

NF-4: The system must efficiently manage concurrent processes. Communications between robot nodes must not cause race conditions or block the main visualization thread.

4.3.3. Usability

NF-5: The application must be easy to use for non-expert users, with clear forms, understandable messages, and simple navigation.

NF-6: The system must validate the entered data and warn of invalid, inconsistent, or out-of-range values.

4.3.4. Maintainability and extensibility

NF-7: The codebase must adhere to OOP principles, ensuring high cohesion and low coupling between the interface, the network logic, and the graphical engine.

NF-8: The code must be kept legible and organized, facilitating its evolution and maintenance, including descriptive comments.

4.4. Test plan

4.4.1. Testing objectives

The main objective of this testing phase is to make sure that simulation system meets all functional and non-functional requirements specified. The testing is meant to identify defects, validate the routing algorithms and guarantee the correct behavior of the GUI and simulation.

4.4.2. Verification Methods

The verification of system requirements will be classified into two methods, following software engineering best practices. This classification will make traceability easier during the implementation phase (6.2. Test implementation):

- **Test (T).** Verification through the execution of the software. It involves providing specific inputs to a module or the complete system and comparing the actual output against the expected result.
- **Review of Design (RoD):** Verification through the passive inspection of the source code, architectural diagrams, or configuration files. This is used for requirements that dictate how the system is built rather than dynamic behaviors.

4.4.3. Scope of testing

To provide an exhaustive validation of the software, the testing strategy is divided into three levels of complexity:

1. **Unit Testing:** Focuses on the smallest testable parts of the application, isolating individual functions to verify them. In this project, unit tests will target the coordinate transformation logic (maps_util.py) and the metric calculations for area completion (ExplorationMap.py).
2. **Integration Testing:** Verifies the correct interaction between different modules. The primary focus will be testing the communication between the UI configuration variables (gui.py) and the central simulation orchestrator (main.py), as well as the JSON file parsing logic.
3. **System Testing:** Evaluates the full integrated application from the perspective of the end-user (Black-Box testing). It involves running full simulation scenarios to validate

the network routing algorithms (BFS), the physical boundaries of the exploration modes, and the UI stability.

4.4.4. Testing environment

All tests will be executed in a controlled local environment to simulate the end-user conditions. The environment mirrors the specifications established in the Technical Manual:

- **Operating system:** Windows 10/11 (64-bit)
- **Framework:** Python 3.x managed via Conda (environment.yml).
- **Key Libraries:** Matplotlib, Rasterio, Pyproj, Tkinter.

Chapter 5. Design

5.1. Architecture design

The system architecture is designed following a modular, decoupled, and event-driven approach to cleanly separate the user configuration, the physics/spatial engine, the network routing logic, and the graphical rendering. This ensures high cohesion within components and low coupling between modules, following modern standards and satisfying the maintainability requirements (NF-7).

The simulation architecture is divided into five functional modules. The relationship and responsibilities of each file are detailed below on **Table 36. Architectural Modules Responsibilities**:

Module	Files	Responsibility Description
User Interface (UI)	gui.py	Handles initial user data entry, validates manual inputs, manages configuration file uploads (.json), and triggers the execution of the main engine.
Centra Simulation Coordination	main.py config.py	Acts as the main engine. It manages the simulation loop, coordinates updates between modules, and holds global configuration constants.
Topographical & Spatial Module	ExplorationMap.py maps_util.py	Responsible for GIS data ingestion (.dt2), coordinate transformation (GPS to UTM), matrix resolution scaling, and tracking the swarm coverage grid.
Decentralized Network Module	RobotNode.py simulation_utils.py	Model individual rovers as independent entities. Manages concurrent multi-threaded UDP sockets, telemetry encoding/decoding, and

		executes the central BFS routing tree.
Graphical Rendering Engine	visualization.py	Manages real-time 2D rendering using Matplotlib, plotting rover coordinates, drawing active network link lines, and updating the coverage heatmap.

Table 36. Architectural Modules Responsibilities

5.1.1. Component Interaction and Data Flow

The operational data flow of the architecture follows a sequential lifecycle, and can be seen on **Figure 6. Information flow between architectural modules**:

1. **Initialization Phase:** The UI module (gui.py) captures and validates data, then passes a unified data package or a JSON path to the Central Orchestrator (main.py).
2. **Environment Setup:** The Orchestrator calls maps_util.py to parse the topography and initializes the array of RobotNode instances, assigning individual UDP ports.
3. **Simulation Loop:** For each time step, robot positions are updated, rovers are prompted to exchange telemetry packets via UDP sockets, calculates the updated BFS network tree, and pushes the new positional and topology state to the Rendering Engine (visualization.py).

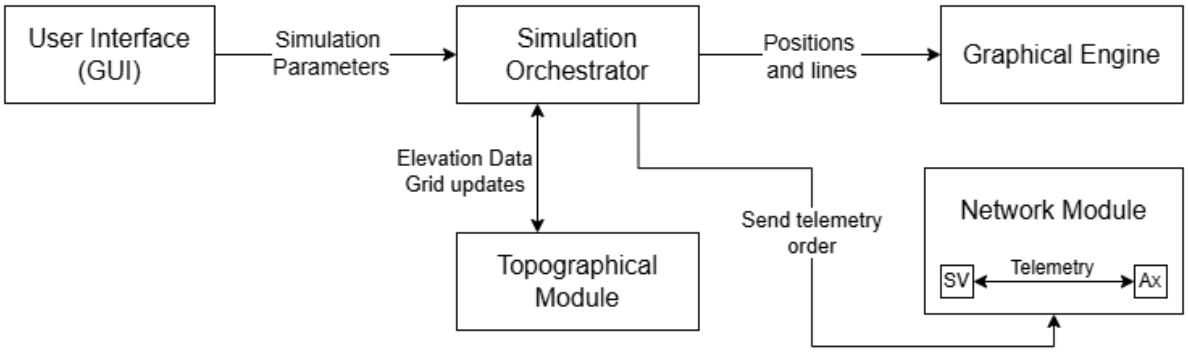

Figure 6. Information flow between architectural modules

5.2. Detail design

While the architecture design defines the macro-structure of the simulation, the detailed design specifies the internal behavior of the individual components, their data structures, and the execution of the algorithms. This section focuses on the decentralized communication model and the dynamic routing logic.

5.2.1. Node Lifecycle and Telemetry Data Model

Each rover in the simulation is instantiated as a RobotNode object, each of them with a non-blocking UDP socket binded to a unique local port.

A critical design decision was to implement a multi-threaded architecture within each node. When one of the nodes is created, the function receiver_loop starts running, and makes the robot to continuously be listening for incoming telemetry or routing commands.

The information exchanged through these sockets is structured as lightweight JSON payloads. This standardizes the communication protocol across the swarm. The fundamental telemetry packet contains the following structure, given as an example **Figure 7. JSON structure for telemetry exchange**:

```
{
  "origin": "A1",
  "target": "SV",
  "read": false,
  "payload": {
    "battery": 85.5,
    "pos": [105.2, -45.0],
    "timestamp": "14:32:01"
  }
}
```

Figure 7. JSON structure for telemetry exchange

The payload right now includes the battery of the rover, the coordinates of position and the timestamp. In a real scenario, this could be changed to data about physical factors like air quality, terrain characteristics, ... This will be further explored on 8.2. Expansions.

Here, we will include a UML sequence diagram (**Figure 8. Information exchange sequence diagram**) to better explain the transmission of messages between robots.

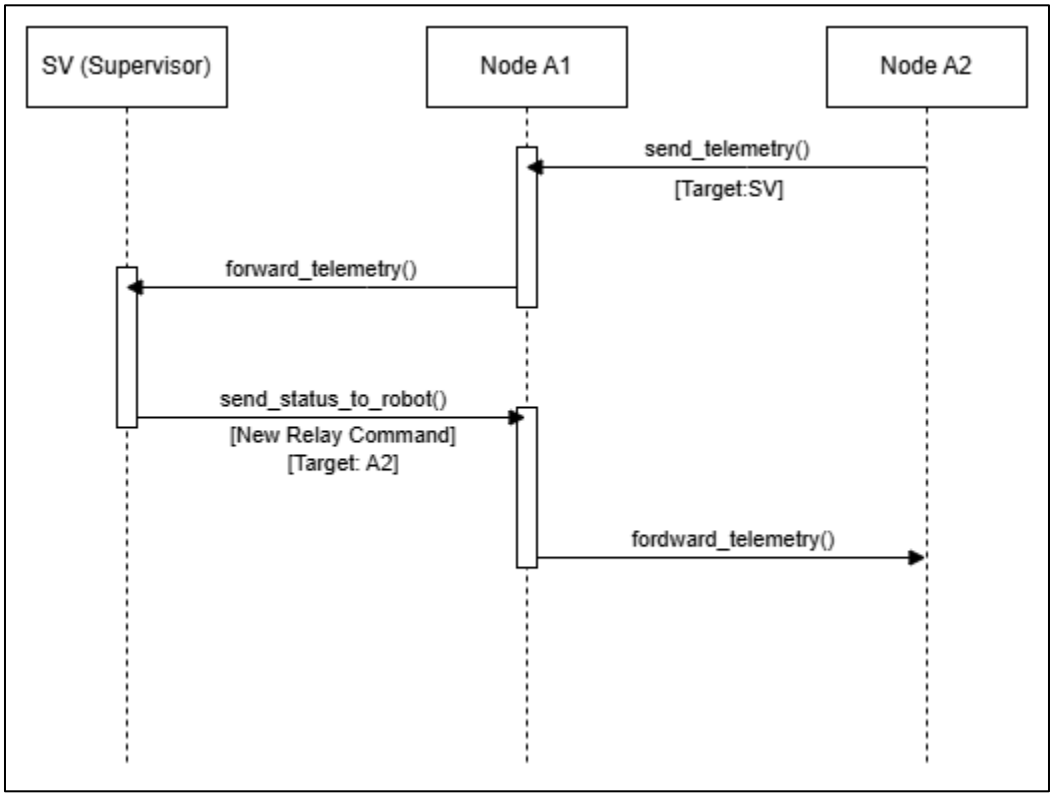

Figure 8. Information exchange sequence diagram

In the diagram, we can see the following steps:

1. The rover A2 sends over their telemetry with final destination SV.
2. Since A2 isn't directly connected to SV, it sends the information to its relay node, A1, which acts as an intermediary between A2 and SV.
3. A1 receives the information, see that itself is not the final destination, so sends the package over to its relay node, in this case SV.
4. SV receives the information, sees that they are the target for the package, reads it, and doesn't forward it to anyone.
5. After seeing the position of A2, SV assigns a new relay and sends over the command.
6. The information travels along the line like before.

In this example, there is only one intermediary, but it would work the same with a multi-hop situation. Each node would just send the package over to their relay, and eventually someone in the line would have SV as a relay.

5.2.2. Dynamic Routing Algorithm (BFS Implementation)

The core networking logic is handled by the build_routing_tree function within the simulation_utils module. Since lunar terrain often blocks direct Line-of-Sight (LoS) communications, explorer nodes must dynamically route their telemetry through intermediate peers to reach the Supervisor (SV) node.

The system calculates this dynamic topology at every time step using a specialized Breadth-First Search (BFS) algorithm (as explained on 4.1.5. Network Routing algorithm (MANET Protocols)). The algorithm evaluates proximity and node health to guarantee the shortest and most reliable multi-hop path. The execution follows these specific rules:

1. **Initialization**: The SV node acts as the root of the BFS queue.
2. **Distance Evaluation**: The algorithm calculates the Euclidean distance between the current node and all unvisited neighbors. Neighbors within the communication threshold (given by the configuration, config.py) are considered candidates.
3. **Health Constraint**: Before assigning a candidate as a valid relay, the algorithm checks its last known battery level. If a node's battery is below 20%, it is discarded as a relay.
4. **Assignment**: Valid candidates are added to the queue, and their active routing port is updated to point to the current node. This propagates outwards until all reachable nodes are mapped.
5. **Validation**: If any active node is left out of the final routing tree, the system triggers a DISCONNECTED state, halting the simulation.

We will add a flowchart (**Figure 9. Routing algorithm flowchart**) to explain this visually:

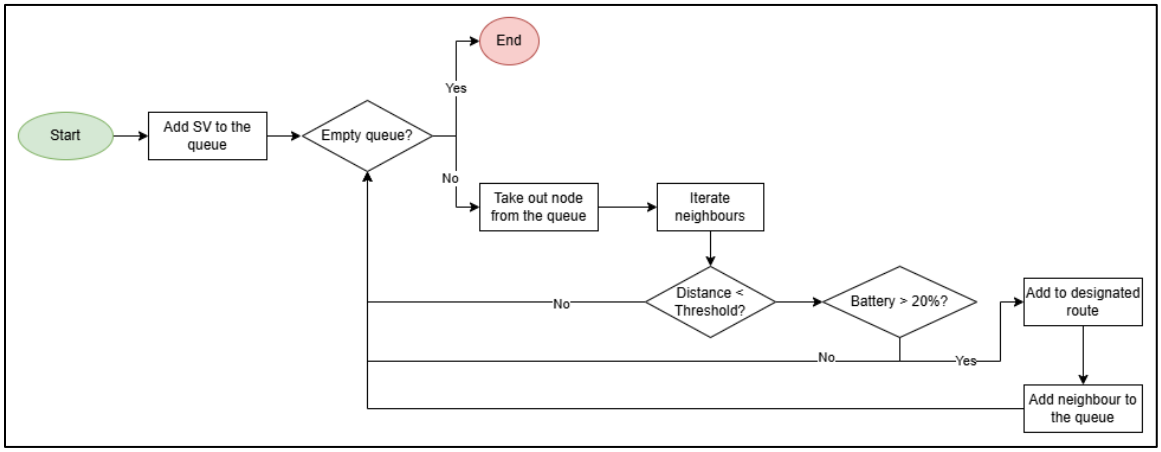

Figure 9. Routing algorithm flowchart

5.2.3. Behavioral Design Patterns: Strategy Pattern

To fulfill the maintainability and extensibility constraints (NF-7), the system implements the Strategy Design Pattern to manage the rovers' movement modes. This behavioral pattern isolates the specific exploration logic from the simulation execution loop, enabling the swarm to change its navigational behavior without requiring modifications to the existing infrastructure.

This allows to separate the rovers and grid, from *how* the simulation behaves. The classic roles of the Strategy pattern are mapped to the codebase as follows and can be seen on **Figure 10. Strategy Pattern UML**:

1. **Strategy (Abstract Base Class):** Represented by the **ExplorationStrategy.py** abstract class. It defines the interface that all concrete exploration modes must meet. It forces the implementation of two core methods: manageMove() (to manage the behavior on the edges) and check_end() (to evaluate termination conditions based on map completion).
2. **Concrete Strategies (Specific Algorithms):** Represented by **FreeExploration** and **FixedExploration**.
    a. **FreeExploration** implements an open-boundary logic where hitting a simulation margin triggers a structural canvas expansion (exp_map.expand), allowing infinite map progression.
    b. **FixedExploration** implements a closed-boundary logic where rovers treat margins as solid walls, when they hit one, the rovers bounce back inside the exploration zone (with a randomized angular noise, to not follow the same straight line that was already explored).
3. **Context (Execution Orchestrator):** In this implementation, the Central Orchestrator (**main.py** via update_robots_movement) acts as the context. It holds a reference to the actice current_strategy instance, calling current_strategy.manageMove() at each time step, not knowing if the swarm is in a free or fixed simulation.

Without this pattern, switching exploration modes would require complex conditional statements directly inside the routing loop or the RobotNode class. This design meets the Open/Closed Principle (OCP) and would be easy to include other exploration modes, for example an AI based exploration, on the future.

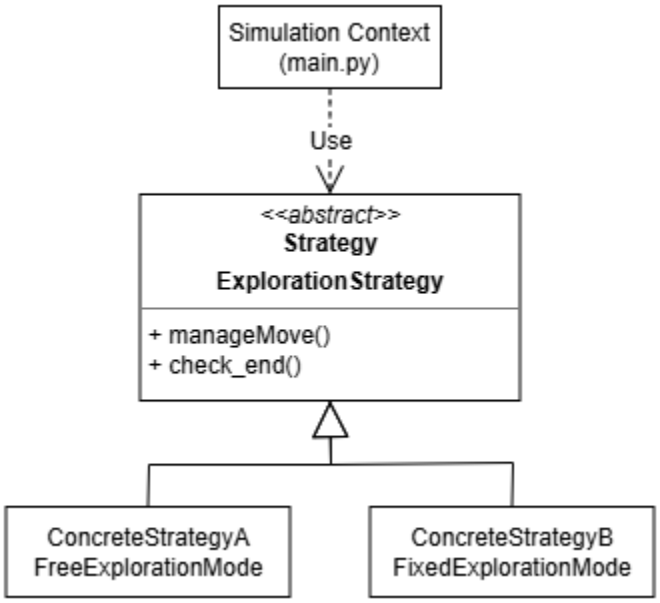

Figure 10. Strategy Pattern UML

5.3. Test design

The test design phase means to translate the test plan (4.4. Test plan) into concrete, actionable and repeatable Test Cases (TC). This section establishes the procedures, inputs, and expected outcomes required to verify the behavior of the system.

5.3.1. Test Case Specification Format

To maintain consistency, all dynamic test cases are documented using a template. Each test case includes a unique identifier (e.g., UT-01 for Unit Test, IT-01 for Integration Test, ST-01 for System Test), the specific objective, the step-by-step execution procedure, the expected result, and the explicit trace to the System Requirements (SR) it verifies.

5.3.2. Test Cases

1. Unit Test Design (UT)

Unit tests are designed to validate isolated internal functions without needing to launch the graphical interface.

Test Case ID	**UT-01: Exploration Completion Percentage Calculation**
Objective	Verify that the mathematical calculation of the explored area gives the correct percentage based on grid cell manipulation.

Prerequisites	Access to the Python testing environment.
Execution Steps	1. Instantiate an ExplorationMap object with a 100x100 grid. 2. Manually set exactly 2,500 cells of the internal grid array to 1 (explored). 3. Call the get_completion_percentage() method.
Expected Result	The method must return exactly 25.0 (representing 25%).
Verified requirements	**SR-3.2.2** (Calculate and show percentage of explored cells).

Table 37. UT-01: Exploration Completion Percentage Calculation

2. Integration Test Design (IT)

Integration tests evaluate the data interchanges between different architectural modules, particularly between the UI and the underlying simulation logic.

Test Case ID	**IT-01: Automated JSON Configuration Override**
Objective	Ensure that uploading a valid .json file correctly parses the data and overrides any manual input present in the UI fields.
Prerequisites	A valid configuration file named test_config.json containing 6 rovers and Free Exploration mode (1).
Execution Steps	1. Launch gui.py. 2. Manually input 4 in the "Number of robots" spinbox. 3. Click "Upload JSON" and select test_config.json. 4. Click "Start Simulation". 5. Observe the initial simulation dashboard parameters.
Expected Result	A warning dialog appears informing the user that manual inputs will be ignored. The simulation launches with 6 rovers in Free Exploration mode, proving the JSON parsed correctly.
Verified requirements	**SR-1.5, SR-1.5.1, SR-1.5.2, SR-1.5.3** (JSON configuration and manual override).

Table 38. IT-01: Automated JSON Configuration Override

3. System Test Design (ST)

System tests evaluate the end-to-end functionality of the software in real-world scenarios.

Test Case ID	**ST-01: Energy Constraint Routing Failure (Battery < 20%)**

Objective	Verify that a rover cannot act as a relay if its battery drops below 20%, forcing the network to disconnect if no alternative path exists.
Prerequisites	Simulation running in Fixed Mode with 3 rovers (SV, A1, A2). A1 is the only relay between A2 and SV.
Execution Steps	1. Wait until A2 moves out of direct range of the SV (distance > 50m). 2. Observe A1 acting as a relay for A2 (dashed colored line). 3. Artificially inject a telemetry payload from A1 with "battery": 15.0. 4. Wait for the next routing calculation turn.
Expected Result	The BFS algorithm discards A1 as a valid relay. Rover A2 loses connection. The simulation stops immediately, displaying a pop-up warning that A2 is disconnected.
Verified requirements	**SR-2.4** (Energy constraint > 20%), **SR-2.3.2** (Stop simulation on connection loss), **SR-2.3.2.1** (Show disconnection message).

Table 39. ST-01: Energy Constraint Routing Failure (Battery < 20%)

Test Case ID	**ST-02: Boundary Collision in Fixed Exploration Mode**
Objective	Validate that the physics engine confines rovers within the initial area during Fixed Mode.
Prerequisites	Simulation launched via manual input. Mode: Fixed Exploration. Area: 100m.
Execution Steps	1. Observe the simulation dashboard until a rover reaches the red bounding box limits. 2. Monitor the rover's velocity vector after contact with the boundary margin.
Expected Result	The rover does not cross the red line. Its velocity vector is inverted and an angular noise is applied, causing it to bounce back into the exploration zone.
Verified requirements	**SR-3.1.2.1** (Stop robots from exiting area), **SR-3.1.2.1.1** (Bounce back with random angle).

Table 40. ST-02: Boundary Collision in Fixed Exploration Mode

5.3.3. Review of Design (RoD) Verifications

As defined in the Test Plan, certain non-functional and architectural requirements tell how the software is built and cannot be verified using dynamic execution tests. These requirements will be verified through inspection and are listed below:

- **SR-2.1 (Independent listening threads):** Verified by reviewing the RobotNode.py source code, confirming the instantiation of threading.Thread targeting a non-blocking socket.SOCK_DGRAM.
- **SR-2.2 (JSON Encapsulation):** Verified by inspecting the send_telemetry function to ensure keys (origin, target, payload, read) are present.
- **NF-1 (Standalone Executable):** Verified by inspecting the output of the PyInstaller build process and confirming the existence of a **.exe** file that executes independently of a Conda environment.
- **NF-7 (OOP Principles):** Verified through the architectural class diagram (4.2.2. Domain data model) and the implementation of the Strategy Design Pattern (5.2.3. Behavioral Design Patterns: Strategy Pattern).

Chapter 6. Implementation

6.1. App structure

The physical organization of the source code was designed to reflect the modular architecture defined in Chapter 5. Design. We isolated utility functions, core classes and test configuration files into separate directories, the repository remains maintainable and scalable.

The final project follows this structure (**Figure 11. Project directory structure**):

```
LunarRobotsComm/
├── main.py
├── gui.py
├── config.py
├── RobotNode.py
├── ExplorationMap.py
├── ExplorationStrategy.py
├── utils/
│   ├── maps_utils.py
│   ├── simulation_utils.py
│   └── visualization.py
├── maps/
│   └── n29_w014_1arc_v3_Caldera_Blanca.dt2
├── arguments/
│   ├── basic_fixed_sim.json
│   ├── basic_free_sim.json
│   └── big_area_free_sim.json
├── environment.yml
└── README.md
```

Figure 11. Project directory structure

Each of the files has the following responsibilities (**Table 41. Files responsibilities**):

File	Responsibility
main.py	Central orchestrator and execution entry point
gui.py	User interface and standalone launcher
config.py	Global constants and threshold definitions

RobotNode.py	Network node class and UDP thread logic
ExplorationMap.py	Spatial matrix and grid management class
ExplorationStrategy.py	Strategy pattern interfaces and algorithms
utils/	
maps_util.py	GIS parser and coordinate transformations
simulation_util.py	BFS routing and swarm movement handlers
visualization.py	Matplotlib rendering and GUI binding
maps/	
n29_w014_1arc_v3_Caldera_Blanca.dt2	Static environment files
arguments/	
basic_fixed_sim.json	Sample configuration file
basic_free_sim.json	Sample configuration file
big_area_free_sim.json	Sample configuration file
environment.yml	Conda environment definition and dependencies
README.md	Repository documentation and execution instructions

Table 41. Files responsibilities

These contents can also be found in the following GitHub repository.

6.1.1. Development Environment Reproducibility

This project utilizes the Conda package management system. Its use of Geospatial Information System (GIS) libraries such as *rasterio*, makes standard options insufficient.

The **environment.yml** file contains the exact snapshot of the development environment, including the specific Python version and all third-party libraries. This ensures that any future developer or ESA researcher can reproduce the simulation environment with a single command without running into compilation errors.

6.1.2. Standalone Executable Deployment

While the environment.yml handles developer reproducibility, a critical non-functional requirement for this project (NF-1) is the delivery of a standalone executable file for end-users or evaluators who do not have Conda installed.

To achieve this, the application is packaged using **PyInstaller** from within the active Conda environment. The compilation process targets gui.py as the primary entry point.

This deployment strategy avoids the complexity of the Python environment, resulting in a single, portable .exe file that natively launches the Tkinter configuration window and the Matplotlib engine.

A special thing we have to keep in mind is to keep the static map files inside the executable (as specified on NF-2). For this, a custom .spec file is configured to ensure that the static files located in the maps/ directory are kept in the executable package.

6.2. Test implementation

Following the framework explained in the test plan (4.4. Test plan) and the test design (5.3. Test design), this section will document the execution of the dynamic test cases. We will report the results (PASS/FAIL) and show evidence of how the system meets the requirements established.

6.2.1. Unit Testing Execution

The unit tests were executed using Python's *unittest* framework. They were used to validate the internal mathematical algorithms without starting the graphical engine.

Test Case	**UT-01 (Exploration Completion Percentage Calculation)**
Execution Date	May 2026
Evidence	A dedicated testing script (test_logic.py) was created. The test instantiated a mock 100x100 grid and artificially set 2,500 cells as explored. The get_completion_percentage() method returned 25.0, triggering the self.assertEqual confirmation successfully.
Result	PASS

Table 42. UT-01 Execution Results

6.2.2. Integration Testing Execution

Integration testing focused on the data handover between the UI data-entry fields and the JSON parser logic.

Test Case	**IT-01 (Automated JSON Configuration Override)**
Execution Date	May 2026
Evidence	As demonstrated in the figure below, uploading the test_config.json file successfully bypassed the manual inputs, triggering the correct UI

	warning and launching the simulation with the JSON-specified parameters. This can be seen on **Figure 12. IT-01 Manual input ignored** and **Figure 13. IT-01 Simulation Parameters Displayed**
Result	PASS

Table 43. IT-01 Execution Results

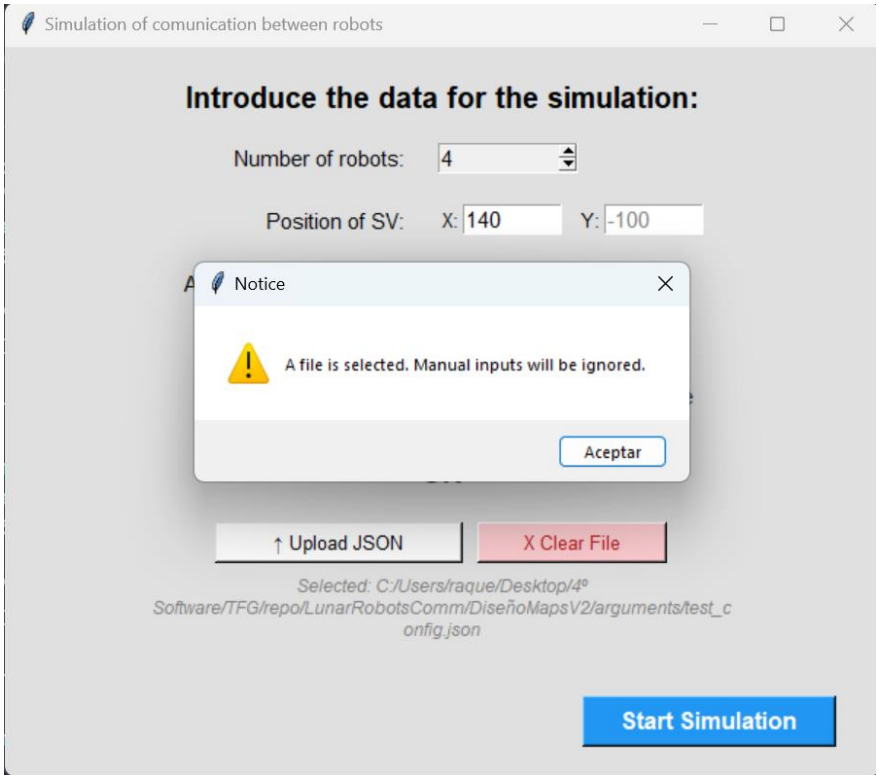

Figure 12. IT-01 Manual input ignored

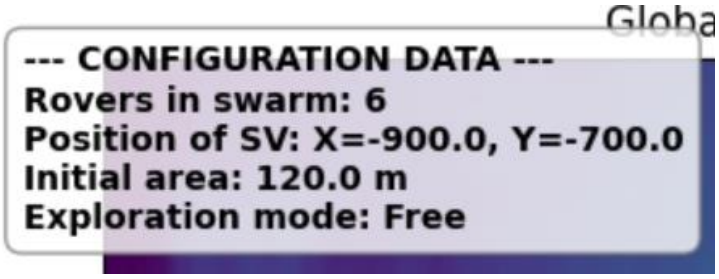

Figure 13. IT-01 Simulation Parameters Displayed

6.2.3. System Testing Execution

System tests evaluated the dynamic physical and network behaviors of the swarm in real-time execution.

Test Case	**ST-01 (Energy Constraint Routing Failure)**
Execution Date	May 2026
Evidence	The simulation was forced into a state where the only available relay node (A1) dropped its battery level below 20%. The BFS algorithm successfully identified the node as unhealthy and discarded the route. Since no alternative path to the SV existed, the system triggered the DISCONNECTED state and halted the execution, notifying the user. This can be seen on **Figure 14. ST-01 Connection lost**.
Result	PASS

Table 44. ST-01 Execution Results

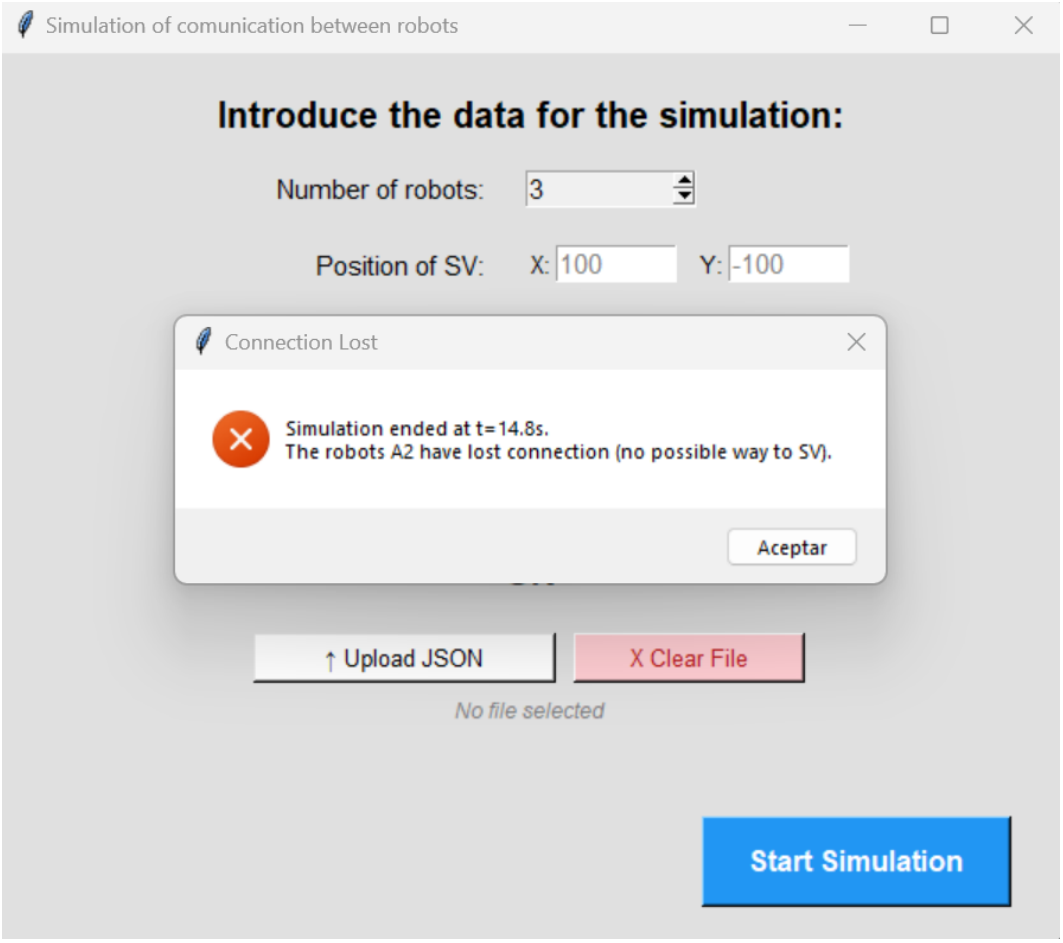

Figure 14. ST-01 Connection lost

Test Case	**ST-02 (Boundary Collision in Fixed Exploration Mode)**
Execution Date	May 2026
Evidence	The rovers were observed reaching the red bounding box limits during Fixed Mode. The collision physics successfully inverted the velocity vectors, applying the required angular noise and keeping all assets strictly within the exploration zone. This can be seen on **Figure 15. ST-02 Edge collision and bounce**.
Result	PASS

Table 45. ST-02 Execution Results

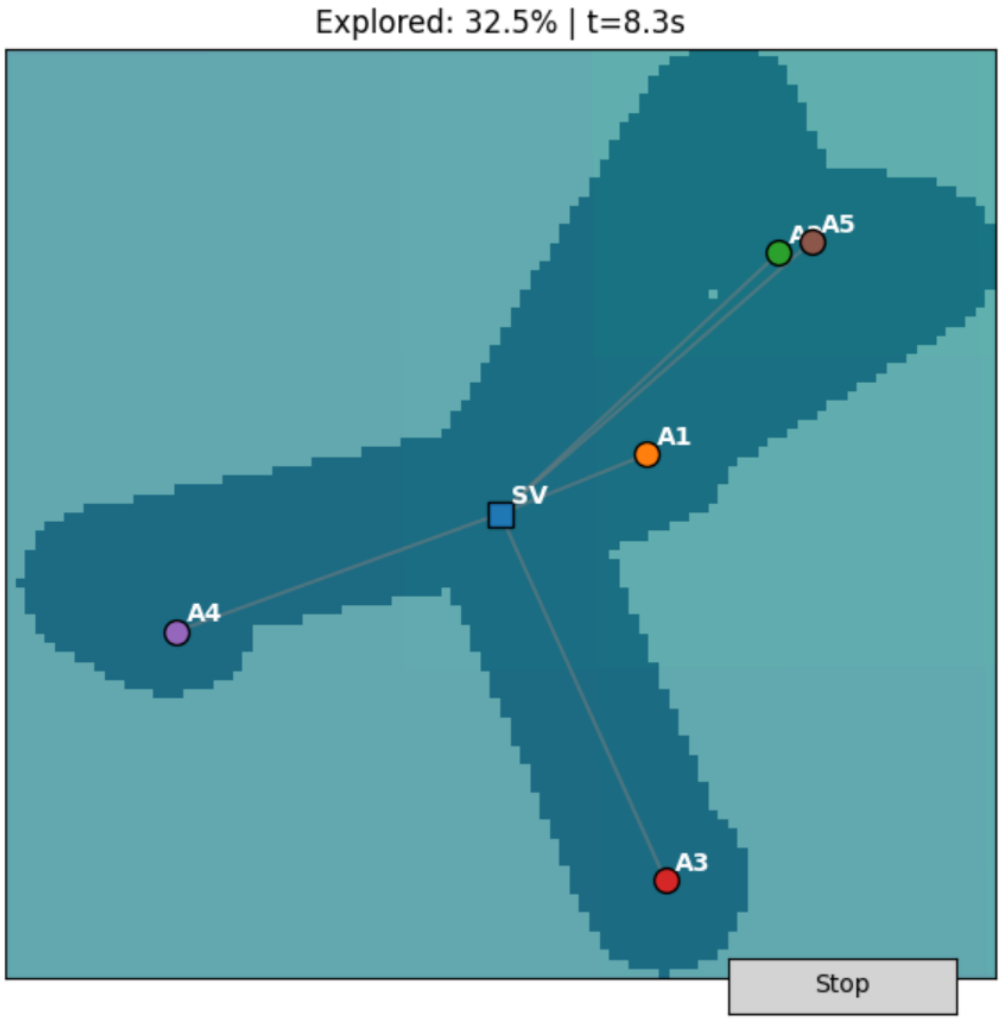

Figure 15. ST-02 Edge collision and bounce

6.2.4. Requirement traceability & verification summary

The following table (**Table 46. Test-Requirements traceability**) details how each critical system requirement was verified (through dynamic testing or passive Review of Design) and links it directly to the specific Test ID or module.

Requirement ID	Requirement Description	Verification Method
SR-1.5	JSON Automated Config	Dynamic Test
SR-2.1	UDP Listening Threads	Review of Design
SR-2.3.2	Stop simulation on connection lost	Dynamic Test
SR-2.4	Energy Constraint > 20%	Dynamic Test
SR-3.1.2	Fixed Boundary Collisions	Dynamic Test
SR-3.2.2	Calculate explored percentage	Dynamic Test
NF-1	Standalone Executable	Review of Design
NF-7	OOP Principles	Review of Design

Table 46. Test-Requirements traceability

6.3. System verification

The objective of system verification is to formally demonstrate that the developed software fulfills all the system and non-functional requirements established in Chapter 4. System requirements.

To achieve this, a Requirements Traceability Matrix (RTM) is utilized (**Table 47. Requirements Traceability Matrix (RTM) system verification**). This matrix maps each original requirement to the specific software component, class, or architectural decision that satisfies it.

Requirement ID	Goal	Verification & Implementation	Status
Introducing simulation parameters			
SR-1.1	Number of rovers input	Implemented in gui.py utilizing a *Tkinter* Spinbox (restricted 3 to 7).	Verified
SR-1.2	SV coordinates input	Implemented in gui.py utilizing *Tkinter* Entry widgets with placeholders.	Verified
SR-1.3	Area size input	Implemented in gui.py via an Entry widget defaulting to 120.	Verified

SR-1.4	Mode selection	Implemented via Tkinter Radiobutton widgets linked to the Strategy Pattern.	Verified
SR-1.5	JSON automated config	Verified in simulation_utils.py (_extract_arguments_file) and gui.py upload logic.	Verified
SR-1.6	Start Simulation Button	Verified in gui.py. The gatherData_and_start function executes validation checks.	Verified
SR-1.7	Simulation information box	Verified in simulation_util.py. The add_info_text function adds a info box to the simulation	Verified
Communication between robots			
SR-2.1	UDP Listening Threads	Implemented in RobotNode.py. Each node binds a socket. SOCK_DGRAM on a daemon thread.	Verified
SR-2.2	JSON Encapsulation	Verified in RobotNode.py. Packets strictly use origin, target, payload, and read.	Verified
SR-2.3	Connection Loss & Relays	Verified in simulation_utils.py. The BFS calculates relays; simulation stops if disconnected triggers.	Verified
SR-2.4	Energy Constraint	Verified in build_routing_tree logic where relay candidates require batt > 20.	Verified
Terrain exploration functioning			
SR-3.1	Exploration Modes	Verified by the decoupling of FreeExploration and FixedExploration classes. (Strategy Design Pattern)	Verified
SR-3.2	2D Grid & Percentage	Satisfied in ExplorationMap.py utilizing a NumPy matrix and get_completion_percentage()	Verified
SR-3.3	Stop/Resume Button	Verified in visualization.py via the Matplotlib Button widget and simulation_state flag.	Verified
Non-functional requirements			

NF-1	Standalone Executable	Verified post-development. Application bundled into a .exe using PyInstaller.	Verified
NF-3	Real-Time Execution	Satisfied in main.py by configuring matplotlib in interactive mode with a 0.1s step.	Verified
NF-7	OOP & Cohesion	Verified through the strict separation of UI, network logic, and Strategy pattern architecture.	Verified

Table 47. Requirements Traceability Matrix (RTM) system verification

Through this traceability matrix, it is confirmed that the current software successfully fulfills the defined scope and is ready for operational testing.

Chapter 7. Manuals

7.1. User Manual

This User Manual provides step-by-step instructions on how to operate the Lunar Swarm Simulation software. The application is designed to not require prior programming knowledge or environment configuration from the end-user.

7.1.1. Launching the application

To start the simulation, simply locate the compiled executable file (lunar_swarm_simulation.exe) provided in the delivery folder and double-click it.

- Note: In the first execution, the operating system may request permission to background network connections. Please allow this to enable the internal UDP telemetry exchange.

7.1.2. Configuration Interface

Once launched, the system will present the Configuration Window (**Figure 16. Configuration Window**). This interface allows you to define the initial parameters of the swarm. You have two operational methods: Manual Input or File Upload.

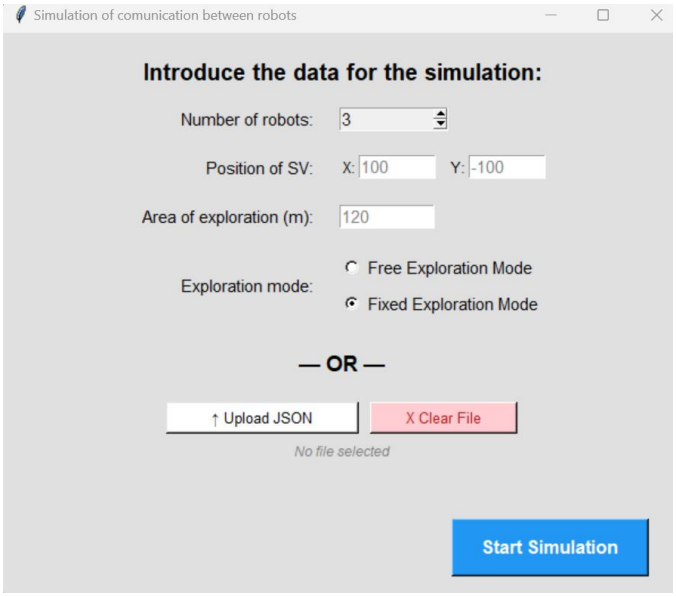

Figure 16. Configuration Window

Method 1. Manual Configuration

Fill in the form (**Figure 17. Manual configuration**) taking into account the following constraints:

- **Number of robots**: Select between 3 and 7 rovers using the arrows.
- **Position of SV** (X, Y): Enter the starting offset coordinates for the Supervisor node relative to the map center. Leave as 0 to start exactly in the center.
- **Area of exploration** (m): Define the size of the initial exploration area (must be a positive number).
- **Exploration mode**: Choose either "Free Exploration" (the map expands as robots reach the borders) or "Fixed Exploration" (robots bounce within a confined area).
- Click **"Start Simulation"**.

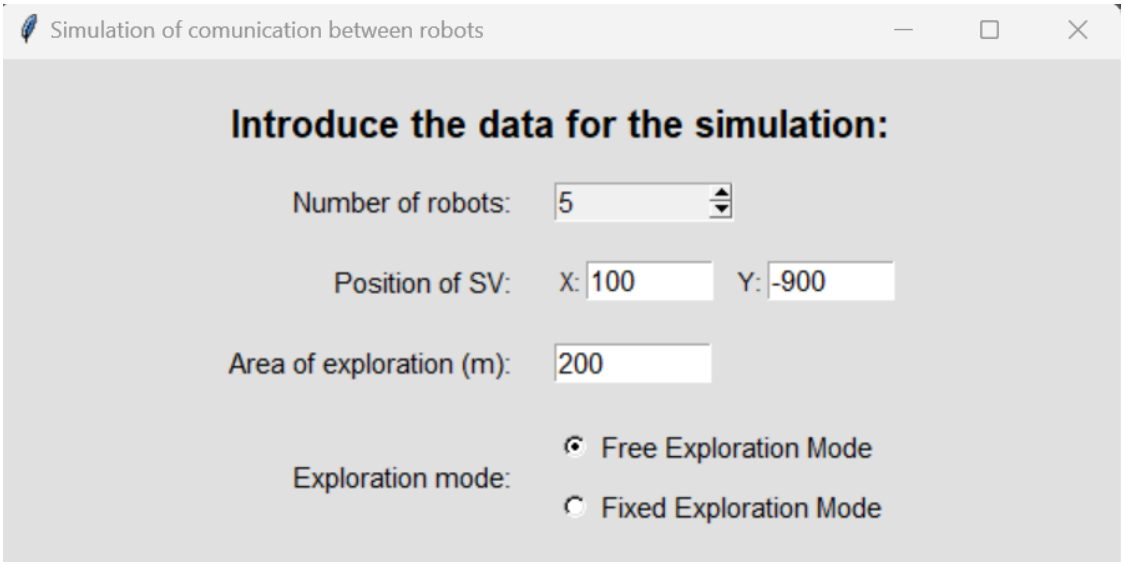

Figure 17. Manual configuration

Method 2. Automated Configuration (.json)

If you wish to load a pre-defined test scenario (**Figure 18. Automated configuration**):

1. Click the "Upload JSON" button.
2. Select a valid configuration file from your computer.
    a. For a guide on how to correctly structure these files, see 7.1.5. Configuration File Schema (.json).
3. The selected path will appear on the screen.

a. Note: Uploading a file automatically overrides any data entered in the manual fields

4. Click "**Start Simulation**".

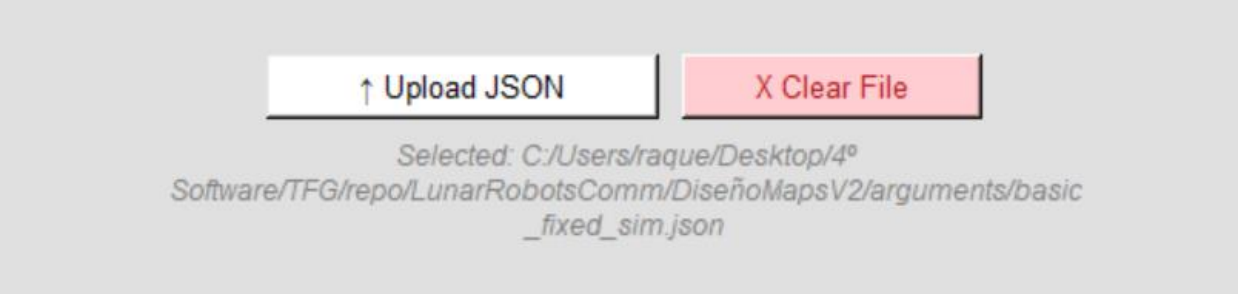

Figure 18. Automated configuration

7.1.3. The simulation Dashboard

Once the configuration is validated, the setup window will close, and the main simulation dashboard (**Figure 19. Simulation dashboard**) will appear in a maximized window. The dashboard is divided into two primary panels:

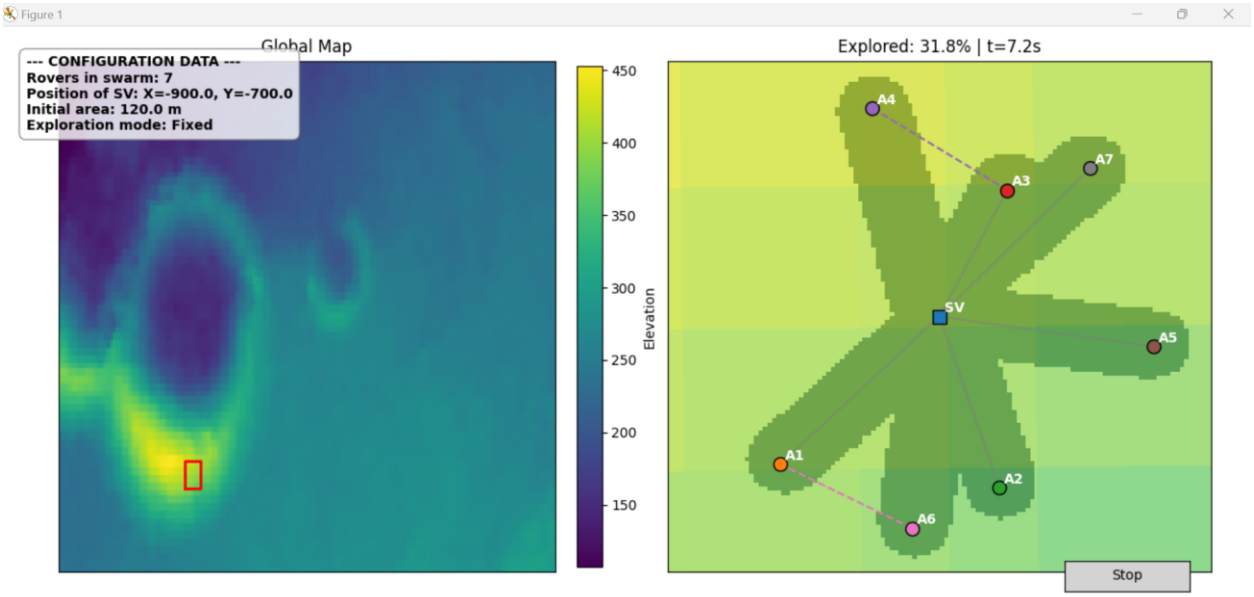

Figure 19. Simulation dashboard

Left Panel. Global Map Reference

This panel displays the static topographical map of the lunar surface we are working on. A red rectangular bounding box indicates the exact region where the swarm is currently operating. If "Free Exploration Mode" was selected, this red box will visibly expand as the rovers discover new terrain.

On the top left of the panel, we can also see the data introduced to perform this simulation.

Right Panel. Live Telemetry and Routing

This panel provides a close-up, real-time view of the swarm's activity:

- **The swarm**: The Supervisor (SV) is represented by a square marker. The explorer rovers (A1, A2...) are represented by circular markers, all including their names.
- **Explored zone**: rovers will leave a visible track as they move, marking the explored zone.
- **Network Topology**: The active communication links are drawn as lines between the nodes.
    - **Gray Lines**. Show connection to the SV node.
    - **Colored Lines**. Indicate that one rover is using another as a relay.

7.1.4. Controls and Simulation End States

During the execution, you can pause the physics and network engines at any time by clicking the "Stop" button located in the bottom right corner. The time, exploration percentage and rovers will stay still.

Clicking it again (now labeled "Resume") will continue the simulation where it left off.

The simulation will conclude automatically and display a pop-up summary message under the following conditions:

1. **Success** (Fixed Mode Only): The swarm successfully explores over 99% of the defined area.
2. **Connection Lost**: One or more rovers have moved too far and no valid relay chain can connect it to SV.
3. **User Cancelation**: The user manually closes the window clicking the 'X' button.

7.1.5. Configuration File Schema (.json)

To automate the simulation setup, the uploaded configuration file must follow the JSON schema. The table below (**Table 48. JSON Configuration File Keys**) specifies the required keys, their expected data types, and valid ranges based on the system constraints:

JSON Key	Data Type	Valid Range / Options
num_rovers	Integer	3 to 7
offset_sv_x	Float/Int	Any number
offset_sv_y	Float/Int	Any number
area_size	Float/Int	Strictly positive (> 0)
exploration_mode	Integer	1 or 2

Table 48. JSON Configuration File Keys

Below is a syntax-compliant example of a .json configuration (**Figure 20. JSON Configuration File Example**) file set up for 5 rovers operating in Fixed Exploration Mode:

```
{
  "num_rovers": 5,
  "offset_sv_x": 100.0,
  "offset_sv_y": -150.0,
  "area_size": 120.0,
  "exploration_mode": 2
}
```

Figure 20. JSON Configuration File Example

7.2. Technical Manual

This Technical Manual is intended for software developers, researchers, or maintainers who wish to explore the source code, modify the simulation logic, or compile a new version of the application.

It will explain the environment setup, execution from the command line interface (CLI), and the deployment used to generate the executable.

7.2.1. System Prerequisites

The host machine must have the following software installed:

- **Git**: For cloning the repository.
- **Conda** (Miniconda or Anaconda distribution): Required for resolving complex C/C++ dependencies associated with geospatial libraries.

7.2.2. Environment Setup

The project includes an **environment.yml** file, which contains an exact snapshot of the required Python version and all third-party dependencies.

To recreate the environment, open Anaconda Prompt, navigate to the root directory of the project, and execute the following commands:

1. Create the environment: *conda env create -f environment.yml*
2. Activate the environment: *conda activate TFG*

7.2.3. Running from Source

Once the environment is active, the simulation can be executed directly from the Python interpreter. The architecture supports two execution methods:

Method 1. GUI Mode (Standard)

To launch the graphical interface, run: *python gui.py*

Method 2. CLI Mode

To bypass the graphical interface and run the simulation directly using a predefined JSON configuration file, use the --config argument with the main orchestrator:

python main.py --config arguments/basic_fixed_sim.json

7.2.4. Building the Standalone Executable

To satisfy the portability requirement (NF-1), the project uses PyInstaller to group everything into a single .exe file.

To generate a new build, ensure the Conda environment is active and run the following command from the project root:

python -m PyInstaller --noconfirm --onefile --windowed --add-data "maps;maps" --collect-all rasterio gui.py

Command Flag Explanation:

- --onefile: Group everything into a single file.
- --windowed: Focuses only on the GUI, not the CLI.
- --add-data “maps;maps”: Makes sure the map file (.dt2) is inside the executable.
- gui.py: Primary entry point of the execution.

After successfully completing the creation, the executable will be located in the new dist/ directory.

Chapter 8. Conclusions and expansions

8.1. Conclusions

As a conclusion, we have seen that this Bachelor Thesis met its goals: designing, developing and verifying a simulation environment made for evaluating the communication and coordination inside a swarm of lunar exploration robots.

Developed within the academic framework of the ESA's COSLE project, this simulation allows us to have a test bed to analyze how assets operate and move under different network and exploration constraints.

Architecturally, in order to replicate real radio transmissions, we integrated multi-threaded non-blocking UDP sockets inside each rover's lifecycle. We also used a central BFS routing algorithm, that allowed the network to self-adjust when one of the rovers is about to lose connection to the supervisor. We tracked this by checking the distance between robots and incorporating health parameters in order to prevent "sick" rovers from acting as relays.

Additionally, we applied a Strategy Design Patter, providing a way for the simulation to switch between exploration behaviors in real time, providing the application with the scalability requested on the requirements.

Finally, the delivery of an executable file containing the simulator, as well as the Conda environment, ensures usability for future research, meeting all the system requirements.

8.2. Expansions

While the current version of the simulation meets all requests, three primary avenues for future development have been identified to increase both the autonomy of the swarm and the physical accuracy of the telecommunication models:

8.2.1. Detailed Telemetry Exchange

As can be seen on 5.2.1. Node Lifecycle and Telemetry Data Model, the payload content sent in the telemetry messages exchanged by rovers right now is just the battery and position of the rover.

In the future, and when performing this exploration on real rovers, this content could be exchanged for more valuable data. For example, some of these rovers can collect the following data:

- Air quality information
- Pictures or videos its surroundings
- Sound
- Mineral composition of the soil [8]

On real simulations, the package could be changed to transport this information.

It is also possible, if other explorations have different requirements, to change the speed of the robots, the exploration radius, or the map in which they move.

8.2.2. AI-Driven Autonomous Exploration

The current movement of the robot is defined by semi-random direction vectors. Future versions of this tool could replace these movements with Artificial Intelligence (AI) pathfinding algorithms.

By incorporating AI, we could save time from the rover wandering aimlessly around the map, instead sending the directly to geographical anomalies (for example craters, ice concentrations, etc.), while maintaining this network structure.

We could transform this simulation into targeted exploration by incorporating multi-agent reinforcement learning (MARL), many agents working together on the same environment [9], or deterministic heuristic search models, that can handle large spaces easily [10].

8.2.3. Transition to Signal Strength-Based Network Disconnection

Currently, the simulation triggers a network disconnection when the distance between 2 rovers is over 50 meters. Going into a more hardware-aligned approach, the next phase will integrate a realistic propagation model based on Wi-Fi 6 (802.1ax) protocols.

The group's current research says that the communications system must guarantee a minimum of 24 Mbit per minute per asset, which requires a minimum speed of 3.2 Mbps.

In a multi-hop network, the data rate degrades as the hop count (n) increases. For a two-hop scenario between an explorer asset and the base station, all individual infrastructure links must sustain a bandwidth greater than 12.8 Mbps.

Wi-Fi 6 MCS Index	Data Rate	Receiver Sensitivity
MCS 0	8.6 Mbps	-90 dBm
MCS 1	17.2 Mbps	-88 dBm
MCS 2	25.8 Mbps	-87 dBm
MCS 3	34.4 Mbps	-83 dBm

Table 49. Wi-Fi 6 Modulation and Coding Schemes (MCS) thresholds

Future software updates will calculate the Received Signal Strength Indicator (RSSI) in real-time, and the system will enforce a disconnection threshold at -85.0 dBm. This ensures that the network operates between the MCS 1 and MCS 2 boundaries (as can be seen on **Table 49. Wi-Fi 6 Modulation and Coding Schemes (MCS) thresholds**), guaranteeing all connections to be between 17.2 Mbps and 25.8 MBPS [11].

This constraint fulfills the 12.8 Mbps requirement explained earlier, preventing data packet drops due to signal fading before physical separation becomes too big.

Chapter 9. Appendices

9.1. Bibliographic references

9.2.1. References

***[1] ROS**: Home*. (n.d.). Ros.org. Retrieved May 16, 2026, from https://www.ros.org/

***[2] Cyberbotics**: Robotics simulation with Webots*. (n.d.). Cyberbotics.com. Retrieved May 16, 2026, from https://cyberbotics.com/

***[3] NetLogo** User Manual*. (n.d.). Northwestern.edu. Retrieved May 16, 2026, from https://ccl.northwestern.edu/netlogo/2.0/docs/whatis.html

[4] A²I roadmap. (n.d.). Esa.int. Retrieved May 17, 2026, from https://esoc.esa.int/a2i-roadmap-0

***[5]** Younis Z A et al. **Mobile Ad Hoc Network in Disaster Area Network Scenario: A Review on Routing Protocols**. International Journal of Online & Biomedical Engineering, 2021, 17(3). (PDF) A multi-objective optimized OLSR routing protocol. Available from: https://www.researchgate.net/publication/380130035_A_multi-objective_optimized_OLSR_routing_protocol [accessed May 14 2026].*

***[6] Open Mesh**: B.A.T.M.A.N. Protocol Concept. (n.d.). Open-Mesh.org. Retrieved May 19, 2026, from https://www.open-mesh.org/projects/open-mesh/wiki/BATMANConcept*

***[7]** Thomas H. Cormen, T. H., Charles E. Leiserson, C. E., Ronald L. Rivest, R. L., & Clifford Stein, C. (2009). **Introduction to algorithms** (3rd ed.). MIT Press. Section VI, Chapter 22, 22.2 Breath-first search (page 613)*

***[8]** Lindsey, L. (2024, April 16). Rover components. **NASA** Science; NASA. https://science.nasa.gov/mission/mars-2020-perseverance/rover-components/*

***[9] An introduction to multi-agents reinforcement learning (MARL)** · hugging face. (n.d.). Huggingface.Co. Retrieved May 20, 2026, from https://huggingface.co/learn/deep-rl-course/en/unit7/introduction-to-marl*

***[10]** Bonet, B., & Geffner, H. (2006). **Learning depth-first search**: A unified approach to heuristic search in deterministic and non-deterministic settings, and its application to MDPs. International Conference on Automated Planning and Scheduling, 142–151. https://cdn.aaai.org/ICAPS/2006/ICAPS06-015.pdf*

[11] ***MCS index*** *- MCS Index Table, modulation and Coding Scheme Index 11n, 11ac, and 11ax*. (n.d.). MCS Index - MCS Index Table, Modulation and Coding Scheme Index 11n, 11ac, and 11ax. Retrieved May 20, 2026, from https://mcsindex.com/

9.2.2. General bibliography

CADRE. (n.d.). NASA Jet Propulsion Laboratory (JPL). Retrieved May 20, 2026, from https://www.jpl.nasa.gov/missions/cadre/

Understand Rapid Spanning Tree Protocol (802.1w). (2025, November 19). Cisco. https://www.cisco.com/c/en/us/support/docs/lan-switching/spanning-tree-protocol/24062-146.html

olsrd: Olsr.org main repository - olsrd v1 - maintained by Freifunk Berlin. (n.d.). https://github.com/OLSR/olsrd

Ramphull D, Mungur A, Armoogum S, et al. A review of mobile ad hoc network (MANET) Protocols andtheir Applications. 2021 5th international conference on intelligent computing and control systems(ICICCS). IEEE, 2021: 204–211

Andreas Mogensen controls ground rover from space. (n.d.). Esa.int. Retrieved May 20, 2026, from https://www.esa.int/ESA_Multimedia/Videos/2015/09/Andreas_Mogensen_controls_ground_rover_from_space

Managing environments — conda 26.5.1.dev9 documentation. (n.d.). Conda.Io. Retrieved May 20, 2026, from https://docs.conda.io/projects/conda/en/latest/user-guide/tasks/manage-environments.html

Using PyInstaller — PyInstaller 4.1 documentation. (n.d.). Pyinstaller.org. Retrieved May 20, 2026, from https://pyinstaller.org/en/v4.1/usage.html

Rasterio: access to geospatial raster data — rasterio 1.4.4 documentation. (n.d.). Readthedocs.Io. Retrieved May 20, 2026, from https://rasterio.readthedocs.io/en/stable/

Carney, S. (2024, March 22). Mars exploration rovers: Spirit and Opportunity. NASA Science; NASA. https://science.nasa.gov/mission/mars-exploration-rovers-spirit-and-opportunity/

Understanding RSSI. (n.d.). Oscium.com. Retrieved May 20, 2026, from https://www.oscium.com/training/understanding-rssi/

Building a Coordinate Reference System - pyproj 3.7.2 documentation. (n.d.). Github.Io. Retrieved May 21, 2026, from https://pyproj4.github.io/pyproj/stable/build_crs.html

Project Management Institute (PMI). (2017). Guía de los Fundamentos para la Dirección de Proyectos (Guía del PMBOK®) – Sexta edición.

PMBOK A Guide to the Project Management Body of Knowledge (PMBOK(R) Guide) Fifth Edition [Libro]. - Project Management Institute, Inc. : Project Management Institute, Inc., 2013. - 5th. - ISBN 978-1-935589-67-9.

Guia de Aprendizaje de la asignatura de Dirección y Planificación de Proyectos Informáticos. – Universidad de Oviedo (2022) – Versión 0.093

unittest — Unit testing framework. (n.d.). Python Documentation. Retrieved May 29, 2026, from https://docs.python.org/3/library/unittest.html

Transformer - pyproj 3.7.2 documentation. (n.d.). Github.Io. Retrieved May 29, 2026, from https://pyproj4.github.io/pyproj/stable/api/transformer.html

Plot types — Matplotlib 3.10.9 documentation. (n.d.). Matplotlib.org. Retrieved May 29, 2026, from https://matplotlib.org/stable/plot_types/index.html

matplotlib.pyplot — Matplotlib 3.5.3 documentation. (n.d.). Matplotlib.org. Retrieved May 29, 2026, from https://matplotlib.org/3.5.3/api/_as_gen/matplotlib.pyplot.html

abc — Abstract Base Classes. (n.d.). Python Documentation. Retrieved May 29, 2026, from https://docs.python.org/3/library/abc.html

Abstract classes in python. (2019, January 10). GeeksforGeeks. https://www.geeksforgeeks.org/python/abstract-classes-in-python/

Strategy in Python. (2025, January 1). Refactoring.Guru. https://refactoring.guru/design-patterns/strategy/python/example

9.2.3. Acronyms

These are all acronyms used throughout the project, ordered in alphabetical order, in order to help the text be more readable:

- AI. Artificial Intelligence
- BATMAN. Better Approach to Mobile Ad Hoc Networking
- BFS. Breadth First Search
- COSLE. Communications System for Lunar Surface and Subsurface Exploration
- CLI. Command Line Interface
- CRS. Coordinate Reference System
- DLR. German Aerospace Agency
- ESA. European Space Agency
- GDAL. Geospatial Data Abstraction Library
- GIS. Geographical Information System
- GUI. Graphical User Interface
- IDE. Integrated Development Environment
- MANET. Mobile Ad Hoc Network
- MQTT. Message Queuing Telemetry Transport
- OBS. Organization Breakdown Structure
- OCP. Open-Closed Principle
- OLSR. Optimized Link State Routing Protocol
- OOP. Object Oriented Programming
- PBS. Product Breakdown Structure
- ROS. Robot Operating System
- RSSI. Received Signal Strength Indicator
- SV. Supervisor Rover / Surface Vehicle
- TC. Test Case
- UDP. User Datagram Protocol
- VAT. Value Added Task
- WBS. Work Breakdown Structure

9.3. Risk Management Plan

This appendix establishes the criteria that will be followed for the project's risk management. It defines the working methodology, risk categories, probability and impact scales, the probability and impact matrix, the register format, and the monitoring criteria that will be subsequently used in the risk identification and registration sections.

9.3.1. Methodology

This project consists of 3 steps in terms of risk management. First, we establish the risk management plan. Second, the risk assessment will be performed, including the identification, analysis, and prioritization phases. Finally, risk management will be executed, which includes response planning, resolution, and continuous monitoring during the project's duration.

The initial assessment is performed prior to the development phase and will be updated as necessary.

9.3.2. Roles and Responsibilities

Since this is a Bachelor Thesis developed individually, risk management falls primarily on the project author. Consequently, the author is responsible for identifying risks, analyzing and prioritizing them, updating the risk register, and monitoring them during execution.

The project manager (thesis tutors) can be questioned and can act as an advisor for the response of the risks.

9.3.3. Budget and Schedule for Risk Management

As can be seen on the Final cost budget specification, a 10% of the budget was reserved for contingencies, just in case a risk manifests.

In case of the initial schedule, no time for contingencies was kept, but the review of the risk status will be conducted at the closure of each major project milestone, where an extraordinary decision can be made.

9.3.4. Risk Categories

To facilitate their identification and treatment, the project risks are grouped into the following categories, adapted specifically for the context of this software simulation:

- **Project Management Risks**: Related to planning, effort estimation, scope definition, or the general organization of the work.
- **Product Risks**: Derived from the technologies used, implementation complexity of the network algorithms, or architectural design issues.
- **Organizational or Personal Risks**: Linked to time availability, workload, prior experience with the technological stack, or the need to balance this project with other academic obligations.
- **External Risks**: Associated with factors outside the direct control of the project, such as third-party library deprecations, external hardware failures, or dependencies related to the ESA COSLE framework

9.3.5. Probability Definition

The probability of occurrence for each risk will be evaluated using a five-level scale. To facilitate subsequent quantitative analysis, each level is associated with a numerical value as indicated in the following table (**Table 50. Definition of risk probability**):

Label	Range	Assigned Value
Very Low	[0%-20%]	0.10
Low	[20%-40%]	0.30
Medium	[40%-60%]	0.50
High	[60%-80%]	0.70
Very High	[80%-100%]	0.90

Table 50. Definition of risk probability

9.3.6. Impact Definition

The impact of each risk will be evaluated independently against the primary project objectives: budget, schedule, scope, and quality. A five-level scale (**Table 51. Definition of risk impact**) will be used, also associated with numerical values that will later populate the probability and impact matrix.

Impact Level	Assigned Value
Imperceptible	0.05
Low	0.10
Medium	0.20
High	0.40
Critical	0.80

Table 51. Definition of risk impact

For prioritization purposes, the overall global impact of each risk will be determined by taking the highest of the partial impacts identified across these four dimensions.

9.3.7. Probability and Impact Matrix

Once the probability of occurrence and the global impact of the risk have been evaluated, its overall significance (Risk Score) will be calculated using the following Probability and Impact Matrix (Probability × Impact), **Table 52. Probability and impact risk matrix**:

Probability	Very High	*0,90*	0,05	0,14	0,27	0,50	0,81
	High	*0,70*	0,04	0,11	0,21	0,39	0,63
	Medium	*0,50*	0,03	0,08	0,15	0,28	0,45
	Low	*0,30*	0,02	0,05	0,09	0,17	0,27
	Very Low	*0,10*	0,01	0,02	0,03	0,06	0,09
			0,05	*0,15*	*0,30*	*0,55*	*0,90*
			Imperceptib	Low	Medium	High	Critical
			0,05	*0,15*	*0,30*	*0,55*	*0,90*
			Impact				

Table 52. Probability and impact risk matrix

This matrix will be used to prioritize the identified project risks. Risks with the highest scores will require preferential attention, a planned response strategy (mitigation, avoidance, transfer, or acceptance), and closer monitoring during execution.

9.3.8. Risk register Format

The project's risk register will compile, for each identified risk, at least the following information:

- Risk identifier
- Risk Name / Description
- Category
- Probability
- Impact on budget
- Impact on schedule
- Impact on scope
- Impact on quality
- Global impact
- Final Score
- Strategy for the risk
- Response to the risk

This format ensures a clear separation between the three levels of risk management: defining the general framework, identifying the risks, and their subsequent assessment.

9.3.9. Monitoring and Control

Risks are not considered static elements; they may evolve as the project progresses. Therefore, they will be reviewed periodically and whenever a relevant incident or a significant difference from the initial schedule is detected.

During each review, the author will verify if new risks have emerged, if any previously identified risks have changed in probability or impact, or if any have materialized into actual issues.

9.4. Content provided in the annexes

The provided delivery folder contains the following elements, as can be seen on **Table 53. Contents of delivery folder**:

File / Directory Name	Description
TFG_RaquelSuarez_UO295000.pdf	The main academic document (this paper) explaining the software lifecycle, research, and conclusions.
README.md	A markdown file containing clear, high-level instructions on how to navigate the repository, install dependencies, and execute the software.
01_Executable/	Contains the standalone compiled application (SwarmCommSimulation.exe). This file requires no installation and is intended for immediate end-user evaluation.
02_SourceCode/	The complete, uncompiled Python repository. It includes all architectural modules, the .dt2 elevation maps, the sample configuration files, and the environment.yml required to clone the Conda environment.

Table 53. Contents of delivery folder

These contents can also be found in the following GitHub repository.